

REQUIREMENTS ENGINEERING

Overview and Installation

서강대학교 기술경영전문대학원 박수진
psjdream@sogang.ac.kr

	주제
Day 1	Introduction
	What is Requirements Engineering?
	Requirements Engineering Process
	- Requirements Elicitation
	- Requirements Analysis: Use Case Modeling(실습Session)
Day 2	. Exercise 1. Use Case Specification
	조별 작성 Specification에 대한 Feedback
	. Exercise 2. Use Case Restructuring: Extend/Include Relationship
	- Requirements Analysis: Quality Attribute Scenario(QAS)
	Quality Attribute Workshop(실습 Session)
	. Exercise 3_1: ATM의 QAS 작성
	. Exercise 3_2: Smart Watch의 QAS 작성
	- Requirements Specification
	- Requirements Validation
	- Requirements Management
	- Class Wrap-up

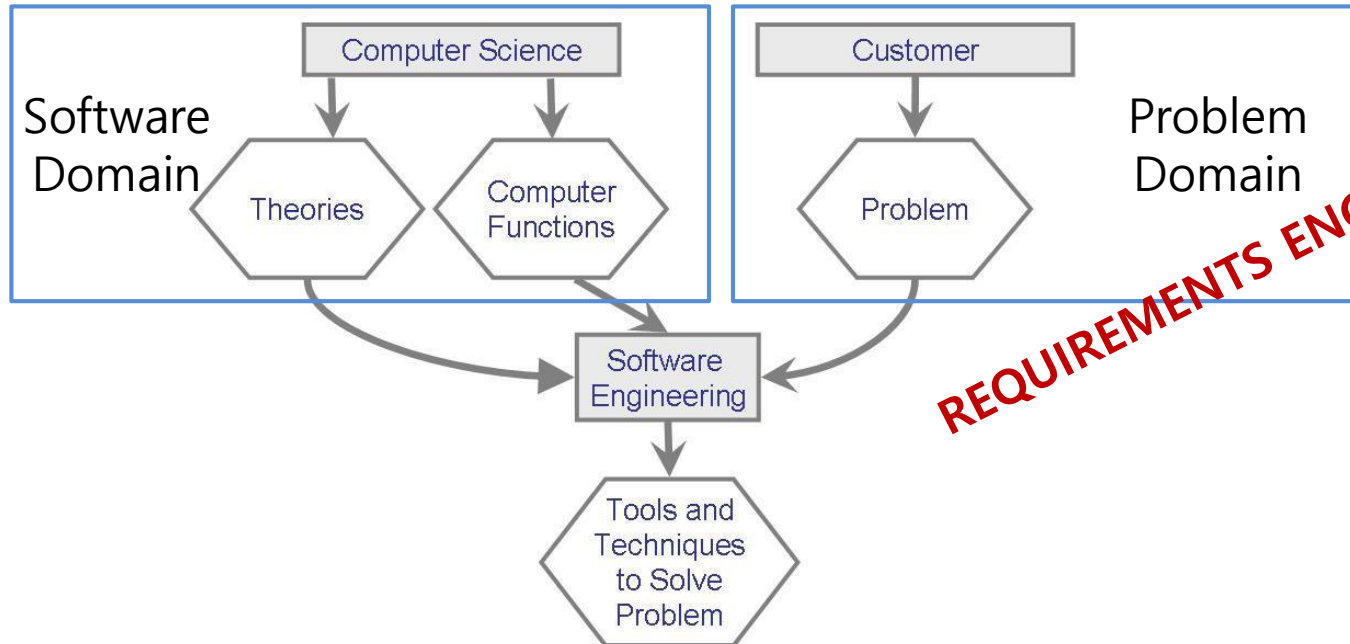
The left side of the slide features a decorative blue background. It includes several overlapping circles, some with diagonal hatching, and a grid of small white dots. A vertical white line with a small circle and dots at its ends is also present.

Part 01

What is Requirements Engineering?

Requirements Engineering

- Relationship between computer science and software engineering & requirements engineering



Requirements Engineering

Quality means fitness-for-purpose.
Cannot say anything about quality
unless you understand **the purpose**.

Not a phase or stage

Communication is as important as
the analysis.

- **Requirements Engineering (RE)** is a set of activities concerned with identifying and communicating the purpose of a software-intensive system, and the contexts in which it will be used. Hence, RE acts as the bridge between the real world needs of users, customers, and other constituencies affected by a software system, and the capabilities and opportunities afforded by software-intensive technologies.

and partly about what is possible

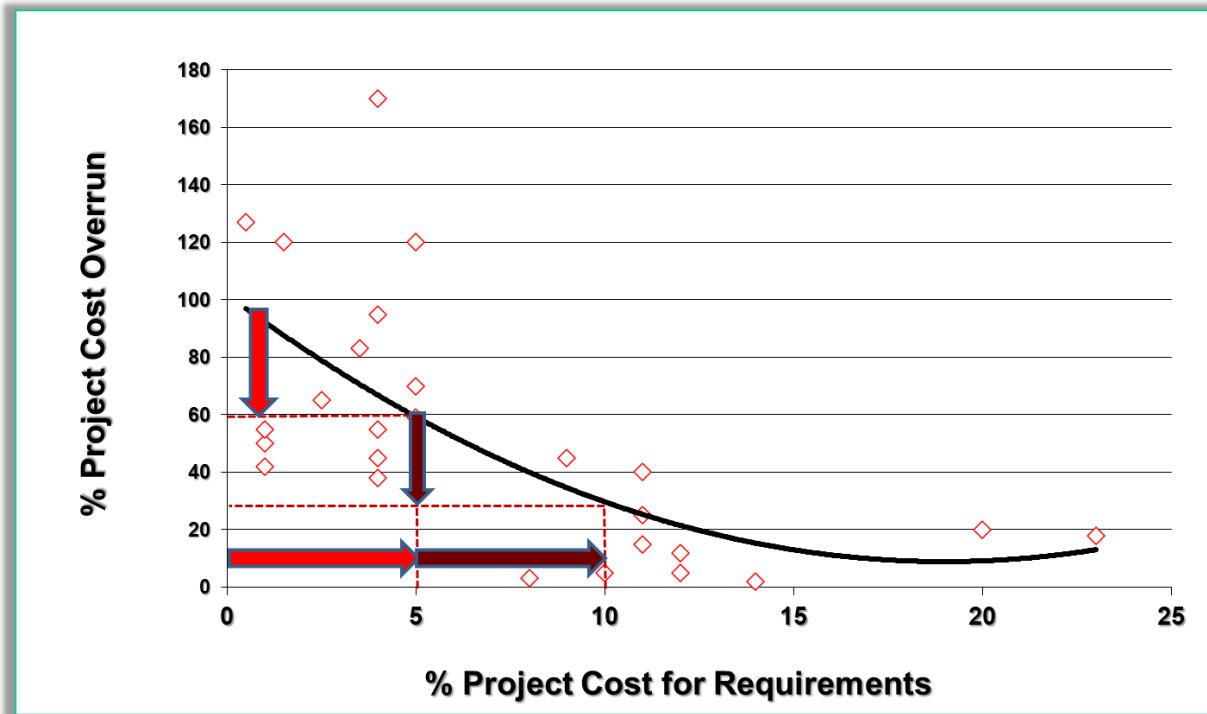
Designers need to know
how and where the system will be used.

Requirements are partly about what is needed.

Need to identify all the stakeholders -
not just the customer and user.

Are Requirements Important?

- **How Much Time** Should We Spend on Requirements?



Are Requirements Important?

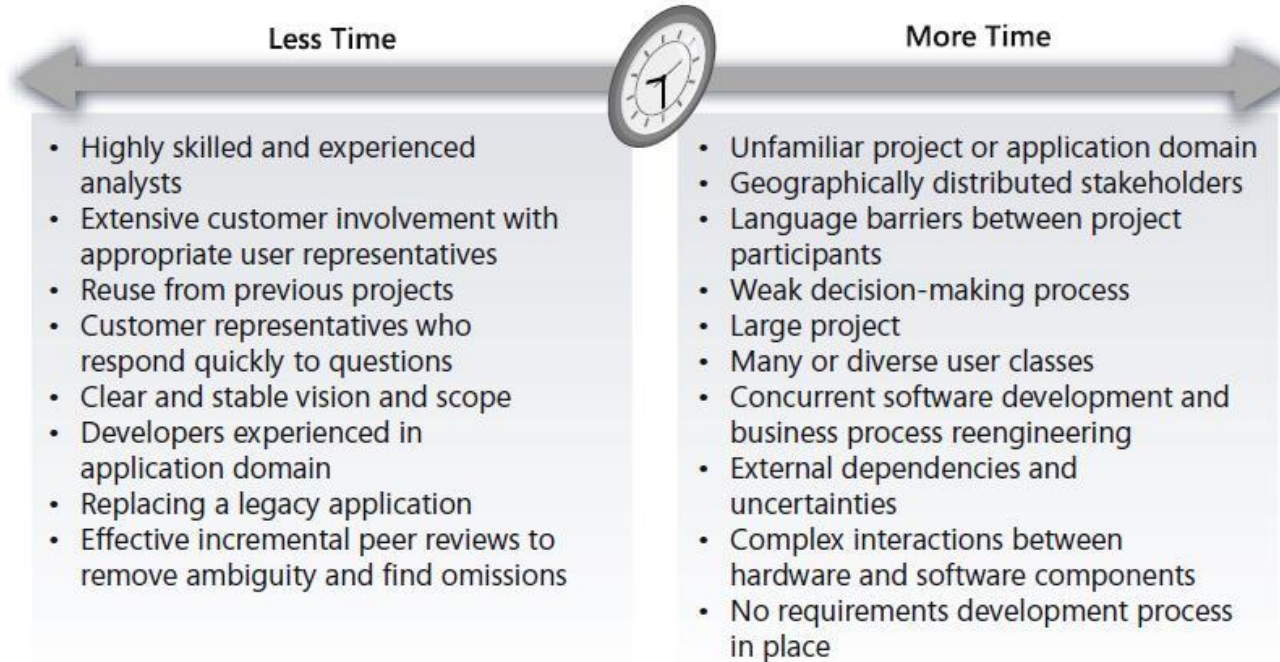
- **How Much Time** Should We Spend on Requirements?

Project Type	Percent of Total Effort on Requirements Development	Months Spent on Requirements Development
Management information systems	3.7	4.44
Systems software	9.0	13.2
Commercial products	7.0	22.7
Military software	10.0	17.5
Outsourced projects	9.0	21.9

<Benchmarks for Requirements Work on Large Project>

Are Requirements Important?

- **How Much Time** Should We Spend on Requirements?



<Factors that influence the time needed for requirements Development>



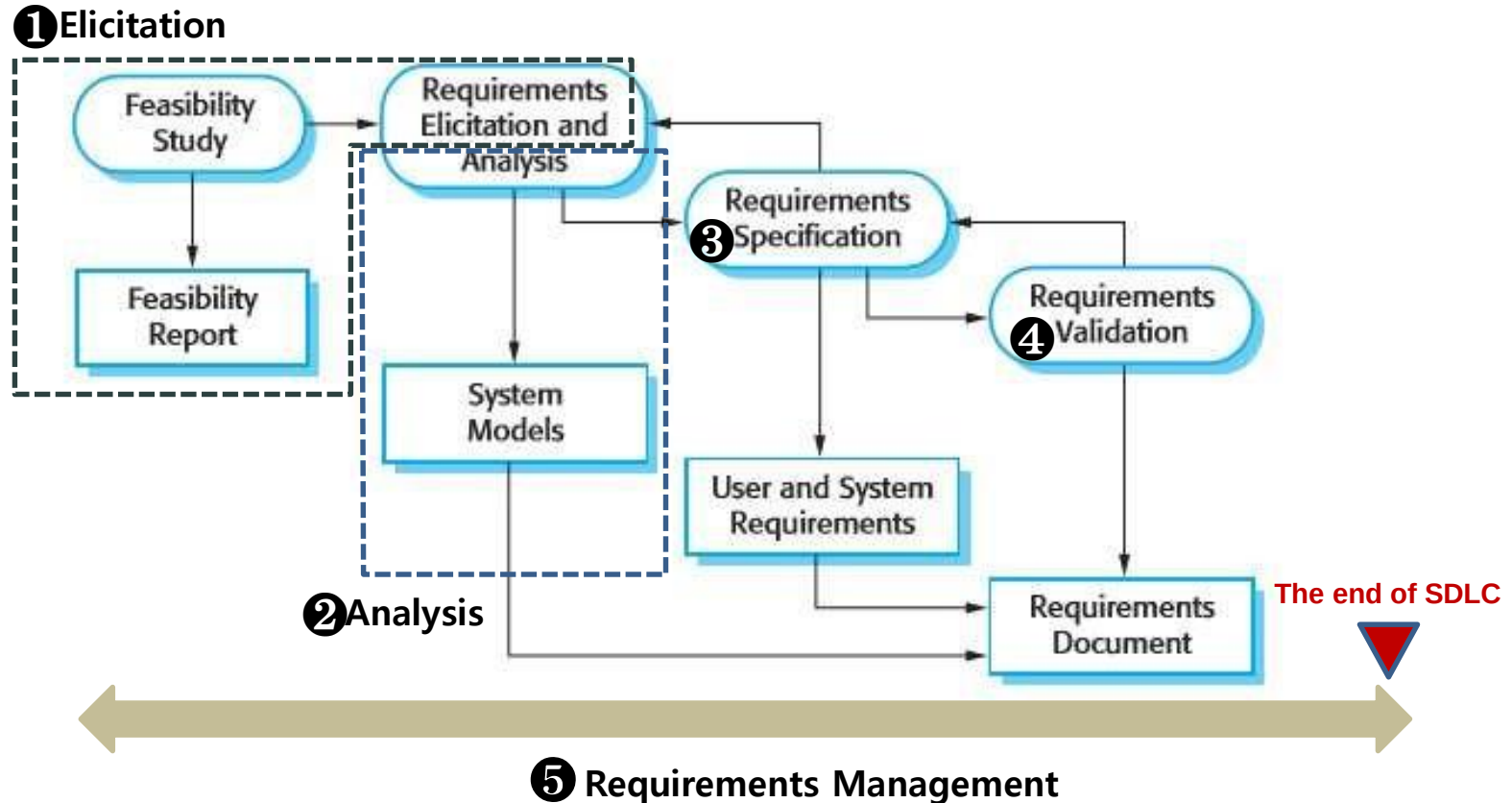
Part 02

Requirements Engineering Process

Requirements Engineering Processes

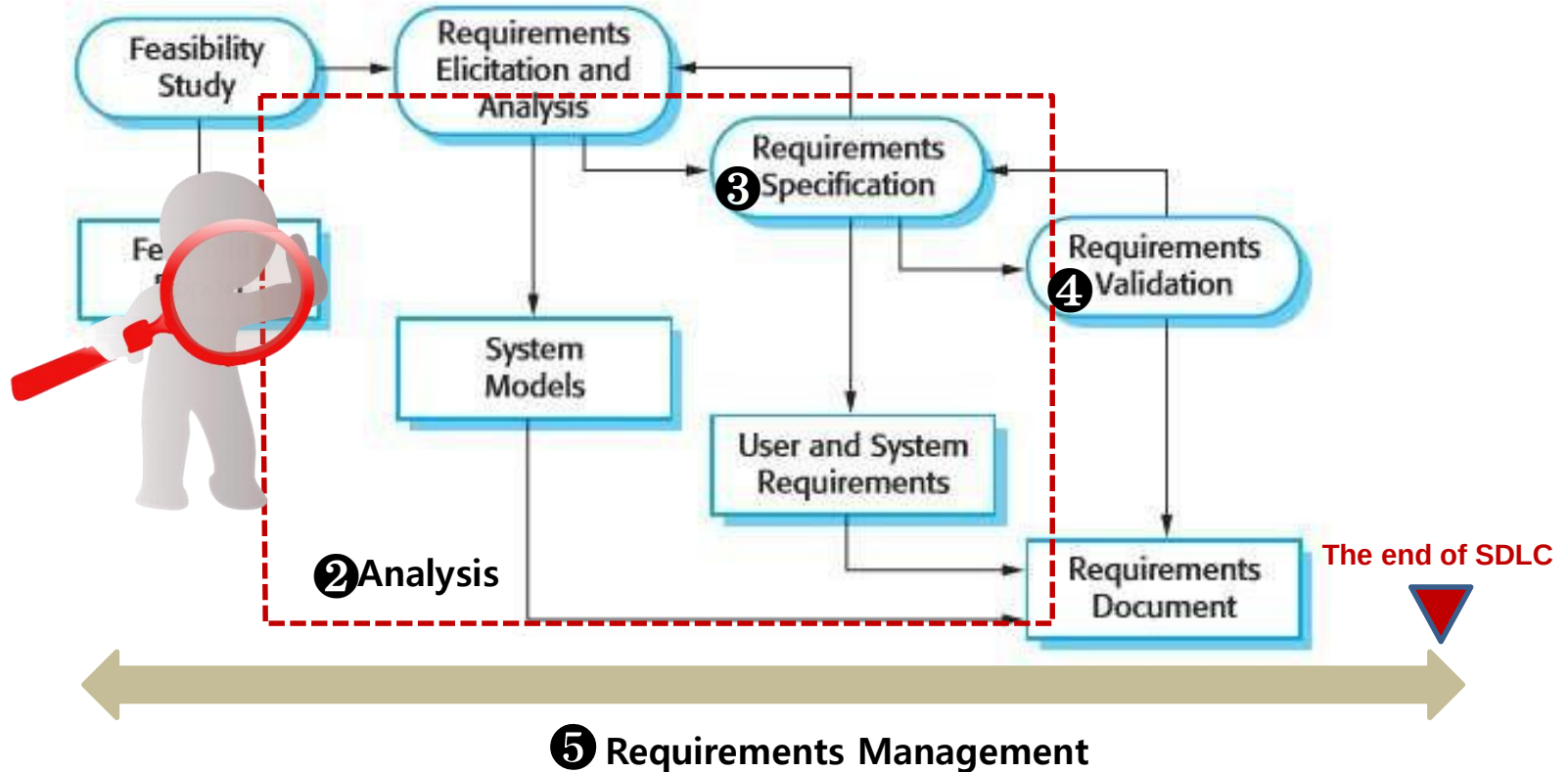
- 5 generic activities common to all processes
 - Requirements elicitation
 - Requirements analysis
 - Requirements specification 요구사항 추출/분석/명세/검증/관리
 - Requirements validation
 - Requirements management

Requirements Engineering Process(Ian Sommerville)

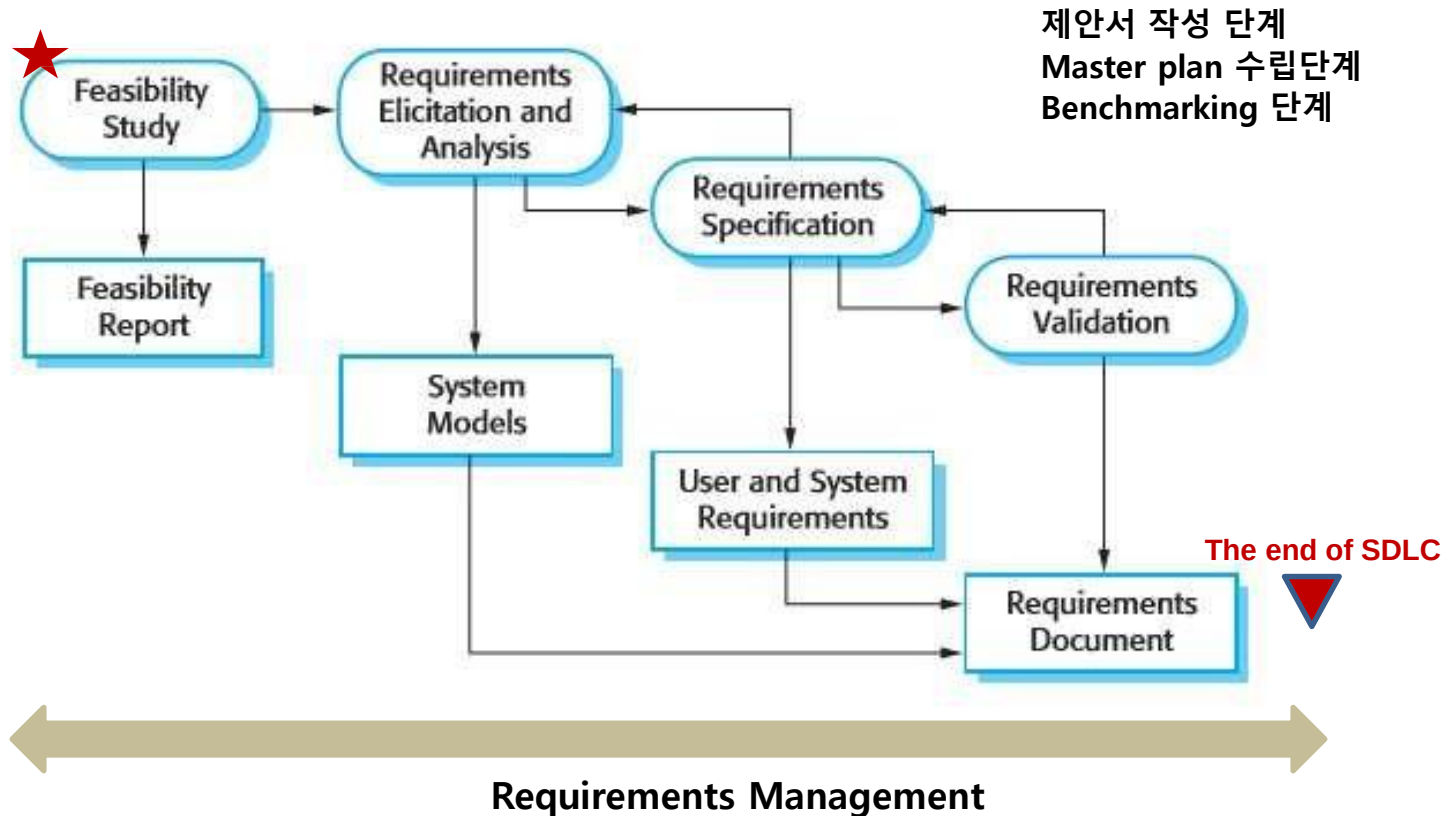


Requirements Engineering Process

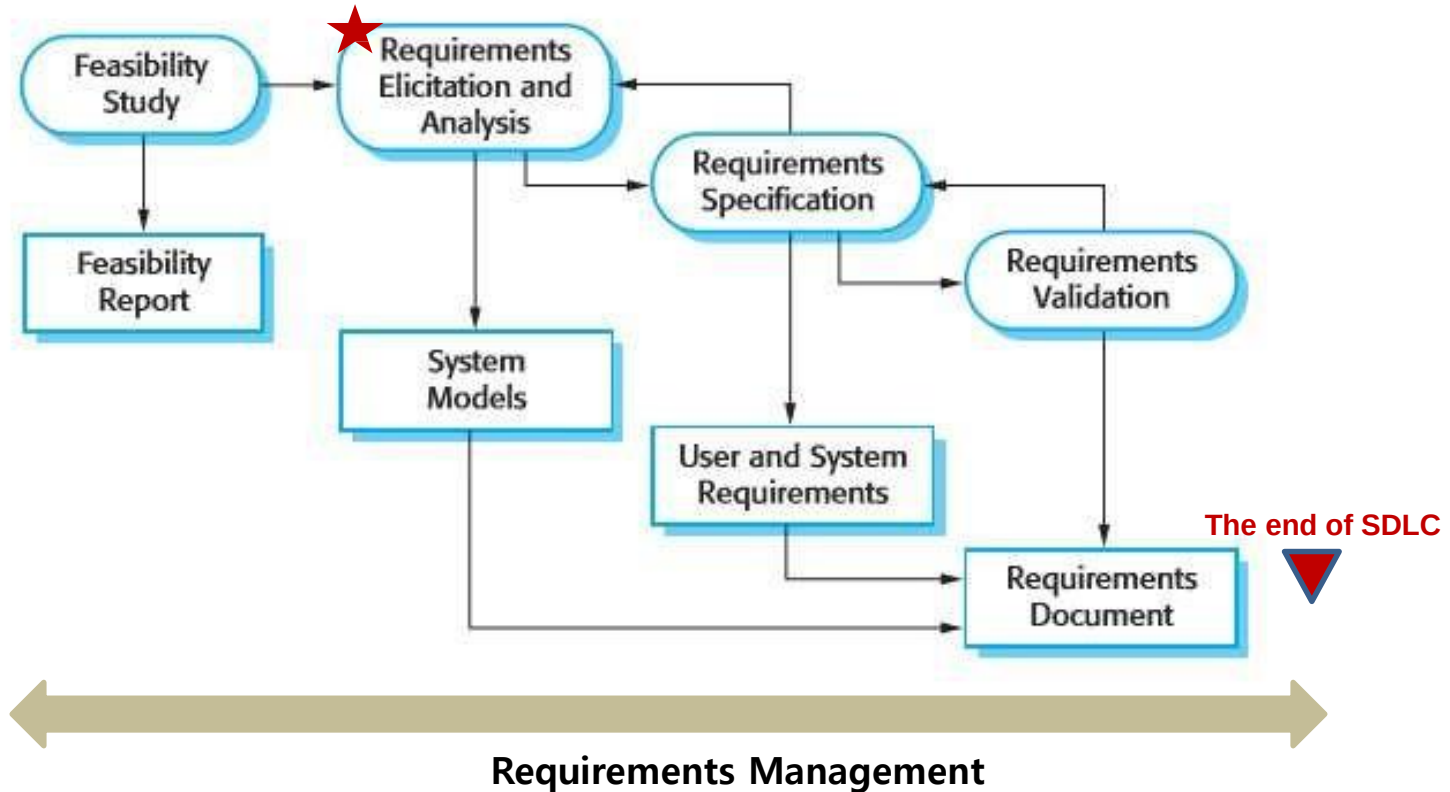
① Elicitation



Requirements Engineering Process



Requirements Engineering Process



The Requirements **Elicitation** and Analysis Activities

1. Requirements Discovery

- Interacting with stakeholders to discover their requirements
- Domain requirements are also discovered at this stage.

2. Requirements Classification and Organization

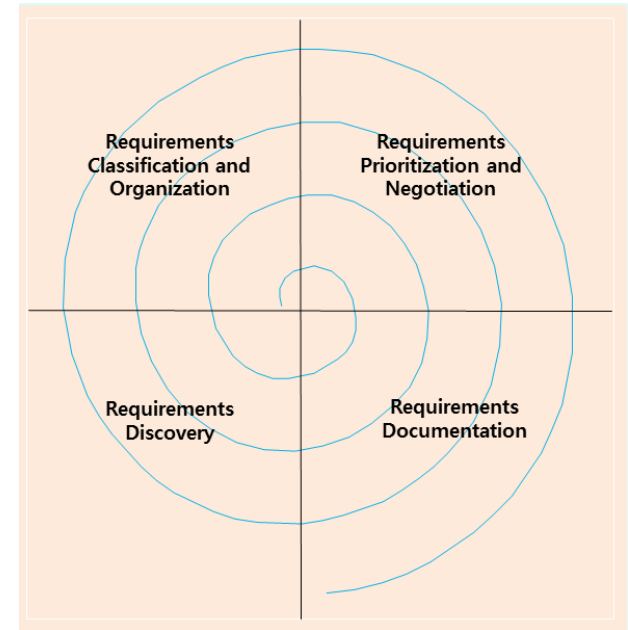
- Groups related requirements and organizes them into coherent clusters

3. Prioritization and Negotiation

- Prioritizing requirements and resolving requirements conflicts

4. Requirements Documentation

- Document requirements
- Input it into the next round of the spiral



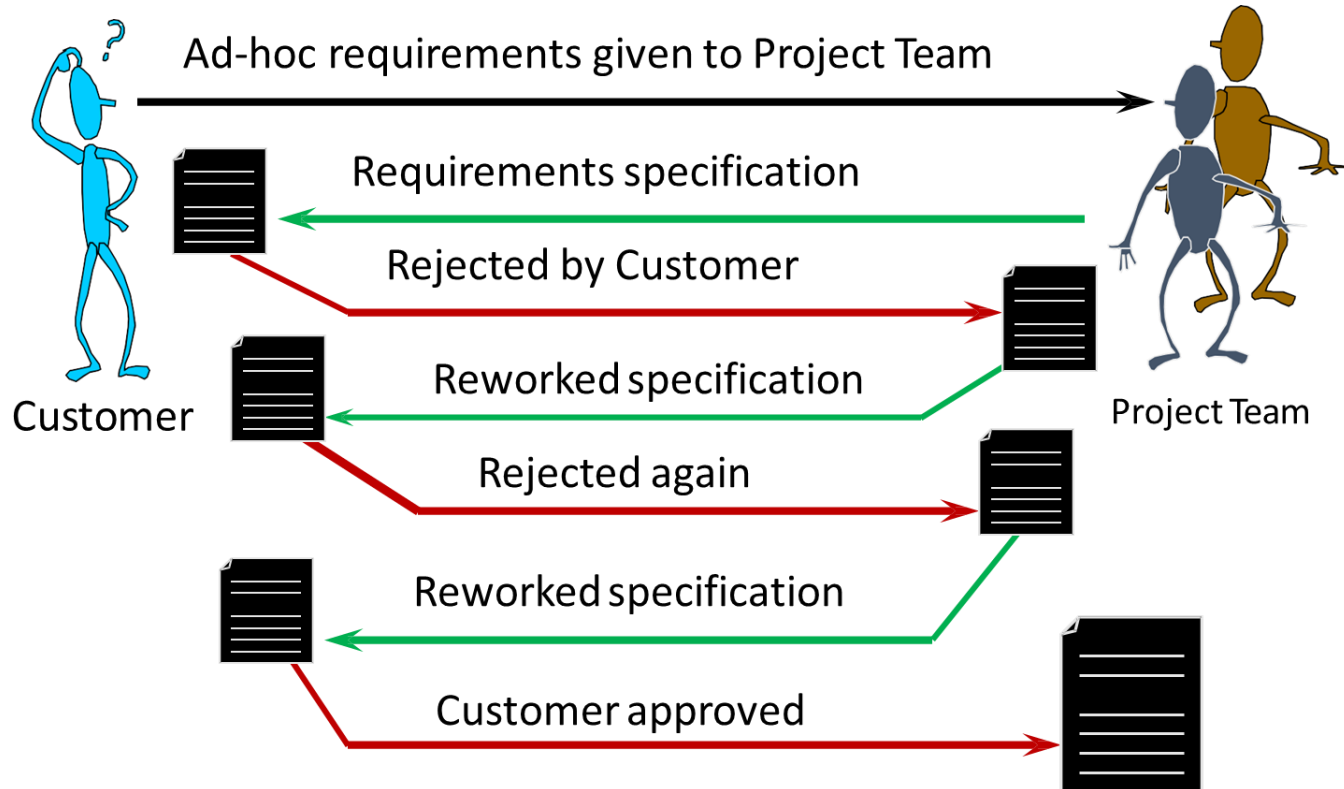
Requirements Elicitation

- There should be a “**problem**” that needs solving.
 - Dissatisfaction with the current state of affairs
 - New business opportunity
 - Potential saving of cost, time, resource usage, etc.
- Collect enough information to **Identify** the “**problem**” and “**opportunity**”
 - Which problem needs to be solved? (identify problem **Boundaries**)
 - Where is the problem? (understand the **Context**/Problem Domain)
 - Whose problem is it? (identify **Stakeholders**)
 - Why does it need solving? (identify the stakeholders’ **Goals**)
 - How might a software system help? (collect some **Scenarios**)
 - When does it need solving? (identify Development **Constraints**: ex. Schedule)
 - What might prevent us solving it? (identify **Feasibility** and **Risk**)

Problems of Requirements Elicitation

- **Vague problem stated by the customer (stakeholders)**
 - Stakeholders **don't know what they really want.**
 - Stakeholders **express requirements in their own terms.**
 - Different stakeholders may have **conflicting requirements.**
 - New stakeholders may emerge and the business environment changes.
- **Organizational and political factors influence the system requirements.**
 - Key man을 Product Champion으로 !!
 - "특정 프로세스 또는 제품이 완전히 개발되고 판매되는 것을 보는 데 과도한 관심(inordinate interest)을 갖는 사람"
- American Marketing Association-
- **The requirements **keep changing** during the analysis process.**

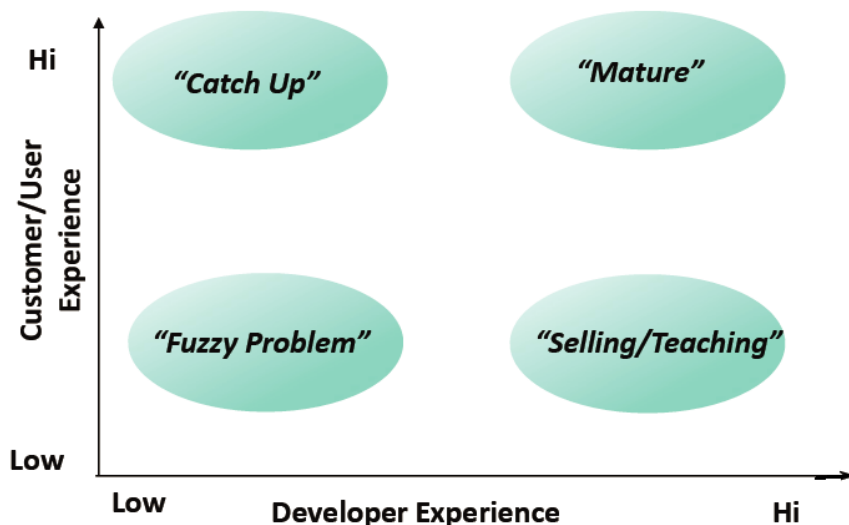
A Typical Situation Encountered



Which Techniques to Use?

- **Appropriate method should be chosen based on:**

- The size and complexity of requirements
- Problem domain
- The number of involved stakeholders



- **Catch Up**

- Role Playing
- Interview
- Requirements Review

- **Fuzzy Problem**

- Requirements Workshops
- Brainstorming
- Storyboards

- **Selling / Teaching**

- Use Case
- Business Modeling

- **Mature**

- Questionnaires
- Prototyping

Example of Elicitation Results

- A list of annotated requirements

ID	Requirement Text
961	No formal training shall be required to operate the RLM.
955	Any new releases or versions of the software shall be sold as new products. Users must p...
954	User software will not be modified or upgraded.
512	The RLM shall return to the refuel location or dump area to within 10 cm of the user-defin...
432	Pressing the screen in an area without a command shall make no sound nor shall it be int...
415	The screen shall be capable of displaying alphanumeric data in blocked, uppercase char...
500	The RLM shall accept lawn and obstacle programming from the user. During programming, th...
321	The RLM shall initiate communications with the GPS through external interface EL-GPS
300	The RLM shall interface with two different external systems, The GPS and the Electronically S...
310	External interfaces include the receipt of location data from GPS and detection of obstacles...
511	The RLM shall not overcut or undercut the border and user defined obstacles by more tha...
510	The RLM shall cut the lawn only within the area defined by the user during the programming.
550	Border programming shall be required to be completed by the user prior to accepting the oth...
411	The Screen shall be 16.25 mm (high) by 105 mm (wide) and capable of displaying two row...
446	Serious errors (for example, blade fouling, Requirement 179) shall not have a button on th...
553	Programming border data shall be terminated by a user request, or when the RLM returns t...
554	After the termination, the RLM shall be ready to receive another command.
418	The screen shall be used to display information from the RLM to the user and accept dire...
561	User shall guide the RLM to the obstacle and indicated that the boundary of obstacle will ...
562	RLM shall record sufficient data (e.g. from GPS) to meet the accuracy requirements stated...
552	RLM shall record sufficient data (e.g. from GPS) to meet accuracy requirements stated in ...
551	User shall guide the RLM to the border of the lawn and indicate that the boundary will be ...



Part 03-1

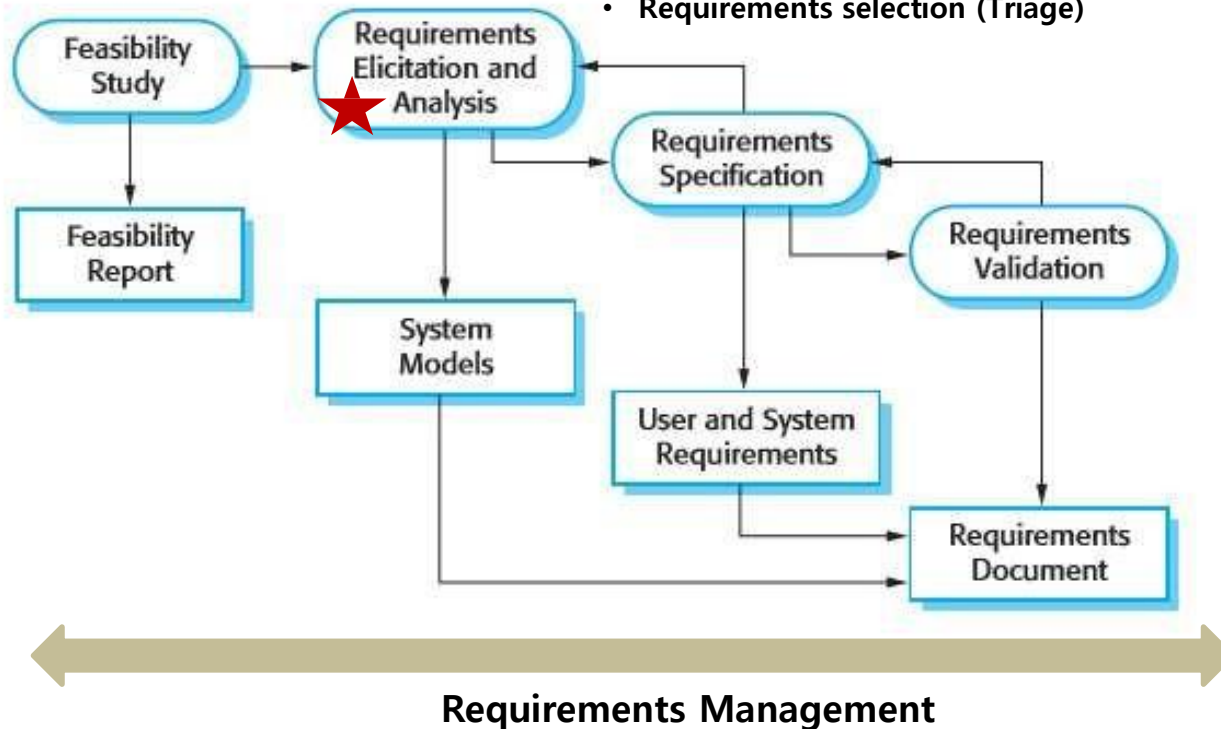
Requirements Analysis

Functional Requirements(FR) vs.
NonFunctional Requirements(NFR)

Requirements Engineering Process

- Different for different systems

- Requirements analysis through requirements models
- Requirements prioritization
- Requirements selection (Triage)



The Requirements Elicitation and **Analysis** Activities

1. Requirements Discovery

- Interacting with stakeholders to discover their requirements
- Domain requirements are also discovered at this stage.

2. Requirements Classification and Organization

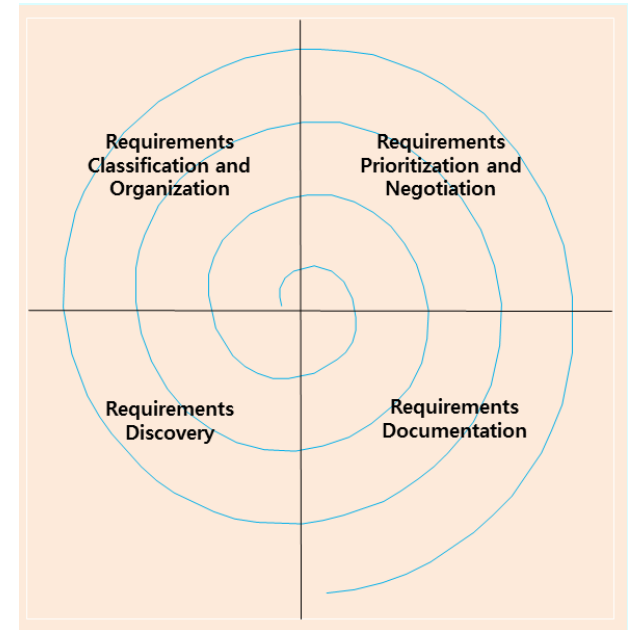
- Groups related requirements and organizes them into coherent clusters

3. Prioritization and Negotiation

- Prioritizing requirements and resolving requirements conflicts

4. Requirements Documentation

- Document requirements
- Input it into the next round of the spiral



The State-of-the-Art Requirements Modeling Methods

1. Structured analysis

- Data Flow Diagram (DFD)
- State Transition Diagram (STD)
- Entity-Relation Diagram (ERD)

2. Use Case analysis

- Use Case Modeling (UC)

Functional Requirements

3. Architectural trade-off analysis

- Quality Attribute Scenario(QAS)

Non-Functional Requirements

4. Goal Analysis

Functional/ Non-Functional Requirements

Functional vs. Non-Functional Requirements

- **Functional requirements**

- Statements of **services** which the system should provide
- How the system should **react to particular inputs**
- How the system should **behave in particular situations**
- May state what the system should **not do**

- **Non-functional requirements**

- **Constraints** on the services or functions offered by the system
 - Timing constraints
 - Constraints on the development process
 - Standards
- Often apply to the system **as a whole** rather than individual features or services

- **Domain requirements**

- Requirements that come from the application domain of the system
- Reflect characteristics of the target domain
- **May be functional or non-functional or the both**

Functional Requirements: Examples

- **Functional requirements for the MHC-PMS**

- A user shall be able to search the appointments lists for all clinics.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.
 - > user authentication에 대한 system의 behavior

* MHC-PMS: Mental Health Care Patient Management System

Non-Functional Requirements

- **Define system properties (and constraints)**
 - Reliability, Response time, Storage requirements
 - Constraints on I/O device capability
 - System representations, Etc.
- **The challenge of NFRs**
 - Hard to model
 - Usually stated informally
 - Often contradictory, difficult to enforce during development
 - Difficult to evaluate for the customer prior to delivery
 - Hard to make them measurable requirements
 - We'd like to state them in a way that we can measure how well they've been met
- **Often called Quality Attributes or Quality Requirements**
 - Often called just the "-ilities"
 - Non-functional requirements may be more critical than functional requirements.
 - If these are not met, the system is totally useless.
- **Critical systems often include non-functional requirements into mandatory requirements.**
 - ex. 의료기기/ 무기제어/ 원자력 발전.. 생명 혹은 자산의 손실을 야기할 수 있는 system들

Classification of Non-Functional Requirements

- Three types of non-functional requirements(Ian Sommerville)
 - **Product requirements**
 - Specify that the delivered product must behave in a particular way
 - e.g., execution speed, reliability, etc.
 - **Organizational requirements**
 - Requirements which are a consequence of [organizational policies and procedures](#)
 - e.g., process standards, implementation requirements, etc.
 - **External requirements**
 - Requirements which arise from [the factors external to the development process](#)
 - e.g. interoperability requirements, legislative requirements, etc.

Non-functional Requirements: Example

- MHC-PMS (Mental Health Care Patient Management System)
 - Product requirements
 - The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day. → Availability
 - The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized. → Usability(But not a good specification)
 - Organizational requirements(제품 자체의 품질과는 무관)
 - Users of the MHC-PMS system shall authenticate themselves using their health authority identity card. → Security



- Each staff member using the system shall be uniquely identified by his or her 8-digit employee number. → security issue 관련 Non-functional Requirements 범주에 넣을 수도 있으나, user authentication에 대한 system의 behavior로 보고 Functional Requirements 범주에 포함 시킬 수도 있음

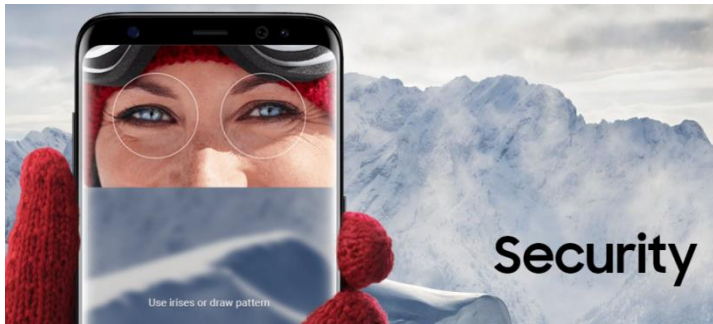
- External requirements
 - The system shall implement patient privacy provisions as set out in HStan-03-2006-priv. → Standard compliance

Non-functional Requirements Implementation

- 홍채인식을 활용한 보안 기능 강화: Security Issue → Non-functional Requirements

업계 관계자는 "갤럭시S8 출시로 생체인증이 다시 활성화될 것으로 기대된다"며 "홍채인식과 공인인증서를 접목한 방식은 안전하면서도 편리한 서비스"라고 말했다. 우리은행 관계자도 "노트7때 선보였던 홍채인증 서비스가 편의성에 힘입어 사용자들의 호응을 얻었다"며 서비스 재개에 대한 기대감을 숨기지 않았다.

반면 보안에 대한 우려는 여전히 존재한다. 아이디와 비밀번호의 경우, 유출이 되더라도 변경할 수 있지만 생체 정보는 바꿀 수가 없다. 이에 삼성전자는 지문과 홍채 등의 생체 정보를 네트워크를 통해 서버로 보내지 않고, 단말기 내부에 저장하는 방식을 택했다. 한국인터넷진흥원 관계자는 "갤럭시S8은 생체 정보를 단말기 내부의 트러스트존에 암호화해 저장하고, 등록된 것과의 비교작업을 거친다"며 "외부로 정보가 유출되지 않아 보안성을 갖췄다"고 말했다.



→ 여러 가지의 Functional Requirements를 파생시킴
→ FR과 NFR이 서로 다른 Track으로 완전히 분리되어 다루어 지지 않음

Functional Requirements vs. Non-functional Requirements

Functional Requirements	Non-functional Requirements
• 내/외부로부터의 stimuli에 대한 System Behavior를 기술함 • 각 requirement가 시스템에 구현되거나(PASS), 혹은 그렇지 못했거나(FAIL), 둘 중 한가지 경우로 판단 가능	• FR이 시스템 상에 implement되었을 때의 expectation level을 기술함 • Quality Attribute별로 시스템이 만족시켜야 하는 range를 만족시켰는지를 검증 가능 → 각 요구사항이 How much/many에 대한 답을 계측 가능한 숫자형태로 명세 되어야 함
• 각 기능적 요구사항은 특정 기능에 국한됨	• 비기능적 요구사항은 시스템 전체 범위에 적용됨
→ 각 기능적 요구사항의 구현은 개발자의 몫	→ 비기능적 요구사항의 구현은 Architect의 몫
→ 기능적 요구사항의 변경 파급효과는 상대적으로 시스템의 특정 컴포넌트(들)에 제한적	→ 비기능적 요구사항의 변경은 전체 시스템의 architecture 변경으로 이어질 수 있음
• 모든 기능적 요구사항을 시스템에 implement하여 만족시키는 것이 1차적 목표 → 시스템의 성쇠를 평가할 수는 없음	• 시스템에 implement된 비기능적 요구사항의 수준에 따라 시스템의 성공과 실패가 결정됨



Part 03-2

Requirements Analysis

Use Case Modeling

The State-of-the-Art Requirements Modeling Methods

1. Structured analysis

- Data Flow Diagram (**DFD**)
- State Transition Diagram (**STD**)
- Entity-Relation Diagram (**ERD**)

2. Use Case analysis

- Use Case Modeling (**UC**)

3. Architectural trade-off analysis

- Quality Attribute Scenario(**QAS**)

Scenario-based Requirements Model

4. Goal Analysis

The State-of-the-Art Requirements Modeling Methods

1. Structured analysis

- Data Flow Diagram (DFD)
- State Transition Diagram (STD)
- Entity-Relation Diagram (ERD)

Scenario-based Requirements Model

2. Use Case analysis

- Use Case Modeling (UC)

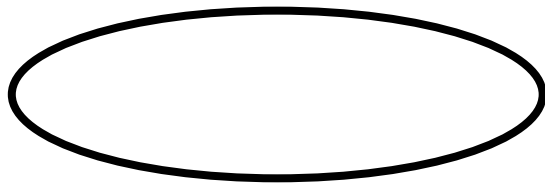
Functional Requirements

3. Architectural trade-off analysis

- Quality Attribute Scenario(QAS)

4. Goal Analysis

Definition of Use Cases

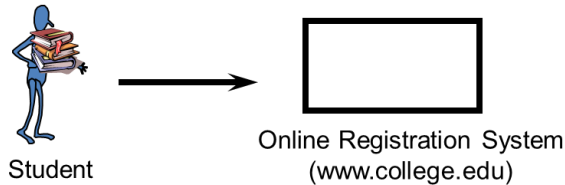
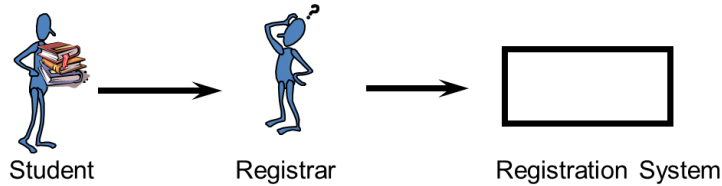


Use Case Name

A use case defines **a sequence of actions**
performed **by a system**
that yields an **observable** result of value
to **an actor**.

Actors and Use Cases

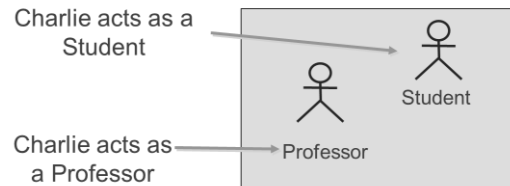
- Actor
 - Someone/something **outside** the system that **interacts with the system**
- Who is pressing the keys (interacting with the system)?



- A user can act as several actors

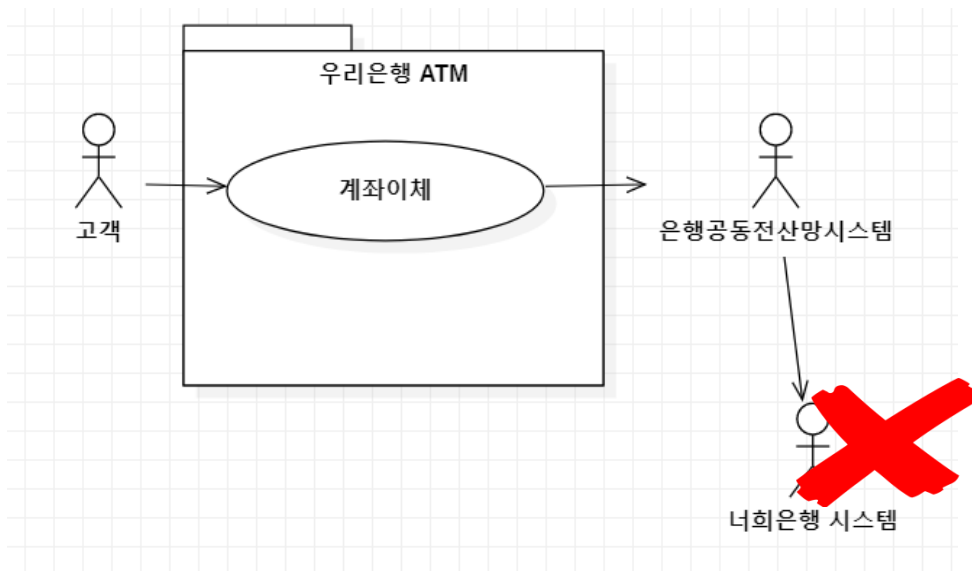


Charlie



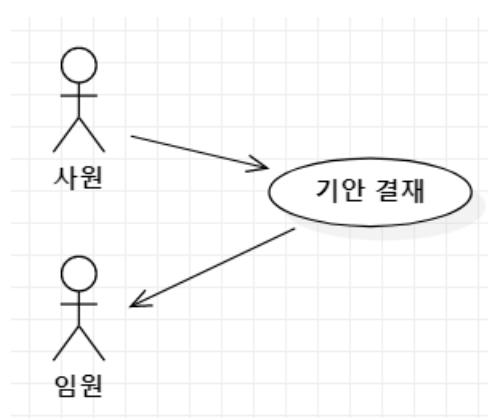
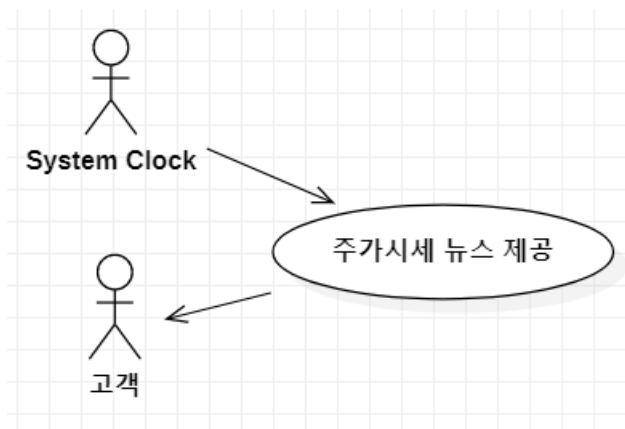
Actors and Use Cases

- Actor
 - Someone/something **outside the system** that interacts with the system
- Something outside the scope of the system that interacts with the system?
→ **None of our business!!!**



Association Navigability between Actors and Use Cases

- Association Navigability = interaction invocation의 방향
- 사람의 역할을 대변하는 Actor는 대부분의 경우, Active Actor
 - Actor → Use Case
- 시스템이 가만히 있는 User에게 Alarm, Request Message, News Feed등을 보내는 경우, 사람의 역할을 대변하는 Actor가 Passive Actor 역할
 - Actor ← Use Case



Database == Actor??

- SW 개발시 기존 DataBase Management System은 legacy 시스템 형태로 그대로 사용한다면, 해당 DBMS는 Actor로 식별

→ DBMS 개발이 SW개발의 범위 바깥에 있음을 명시함

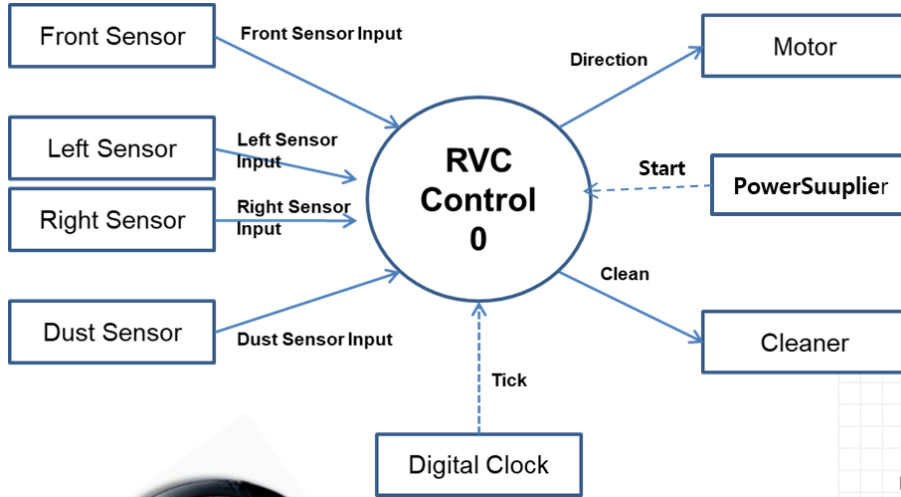


- SW 개발 scope이 시스템에서 운영되는 모든 데이터를 저장하고 조회하는 DBMS(DB) 개발을 포함하고 있다면, Actor로 식별 불가(Actor로 식별된다는 것은 Project Scoping 작업에서 out of scope의 의미)

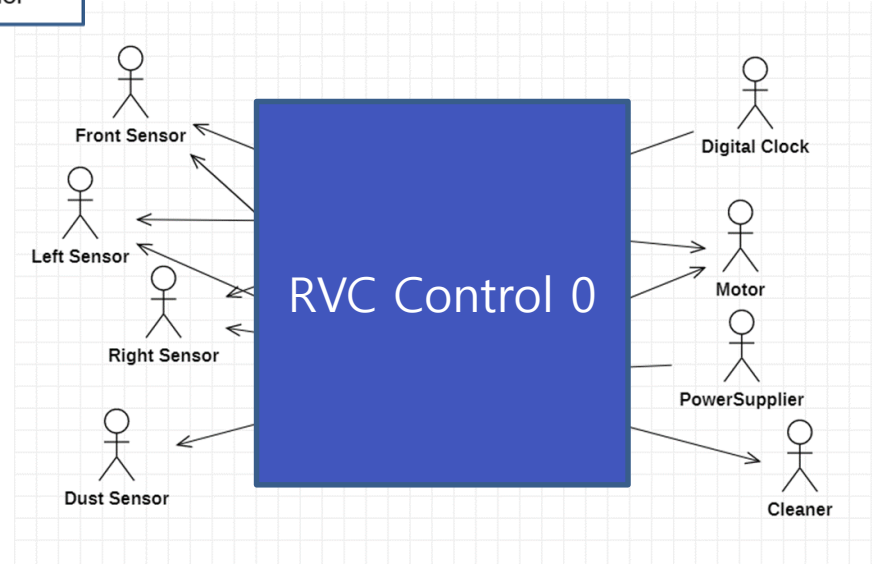


* 과목관련 정보들은 Requirements Specification에서의 명사 → Analysis Class/Attribute
→ Table Column 의 순서로 개발 단계에 따라 refine되어 나감(in the scope)

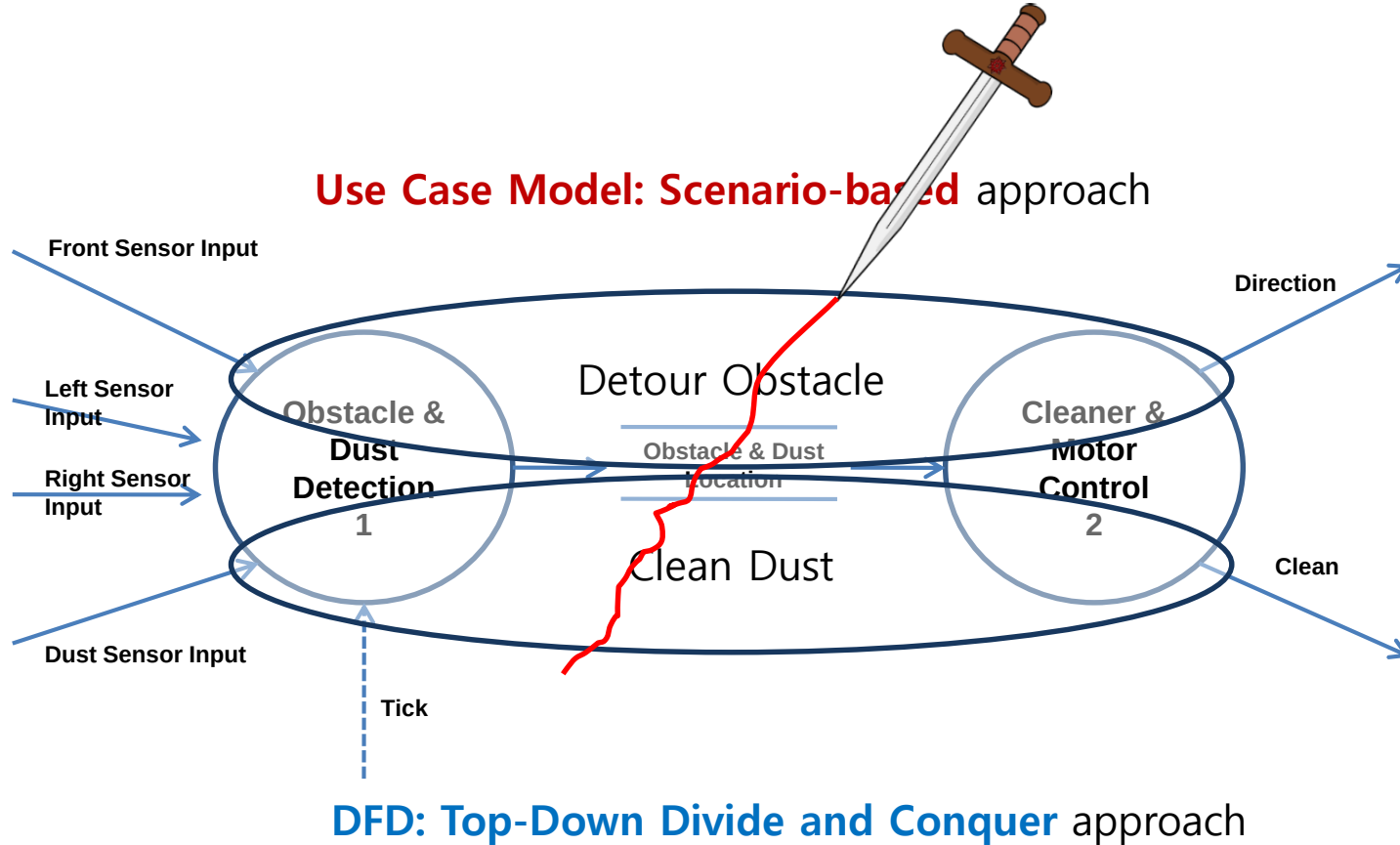
Use Case Diagram vs. DFD(Function Decomposition)



Robot Vacuum Cleaner

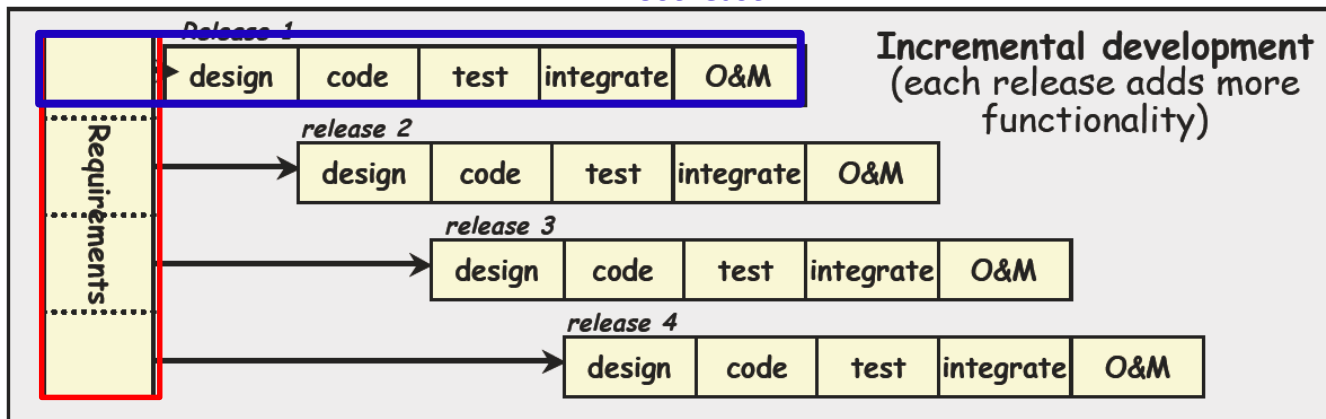


Use Case Diagram vs. DFD(Function Decomposition)

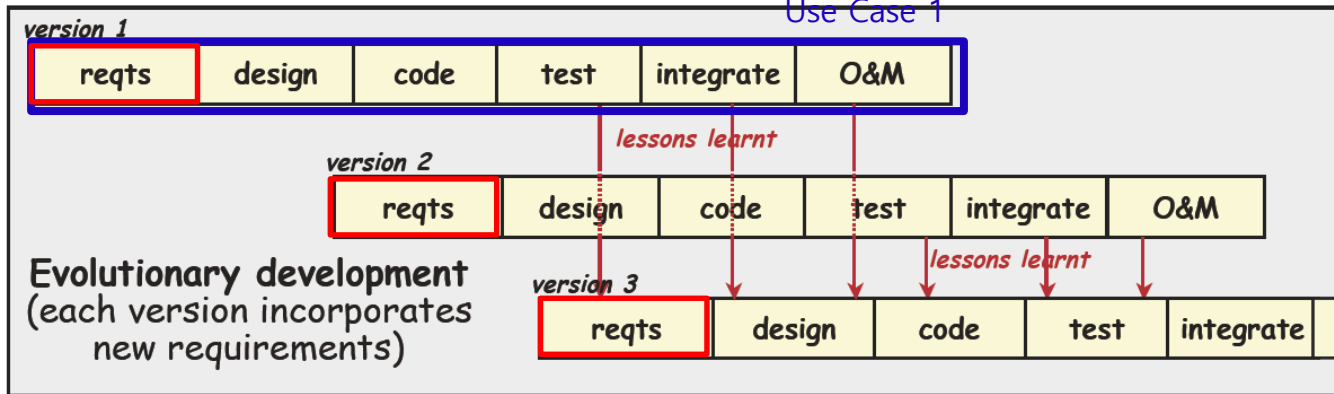


Phased Lifecycle Models Managed with Use Cases

Use Case 1



Use Case 1

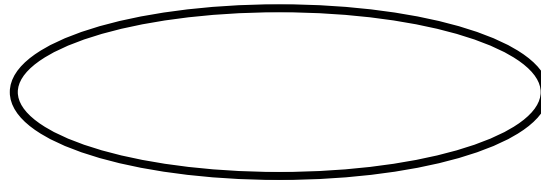


Avoid Functional Decomposition

- **Symptoms**

- Order-relationship among use cases
- Very small use cases
- Too many use cases
- Uses cases with no result of value
- Names with low-level operations
 - "Operation" + "object"
 - "Function" + "data"
 - Example: "Insert Card"
- Difficulty on understanding the overall model

Definition of Use Cases ★★★★★



Use Case Name

A use case defines **a sequence of actions**
performed **by a system**
that yields an **observable** result of value
to **an actor**.

The left side of the slide features a decorative blue background with various geometric elements. At the top left, there is a circle with diagonal white stripes. Below it, a vertical white line contains a small circle and a series of dots. Further down, another circle with diagonal stripes is visible. At the bottom left, a white curved shape contains a solid pink circle and a vertical ellipsis. The right side of the slide is a solid light blue background.

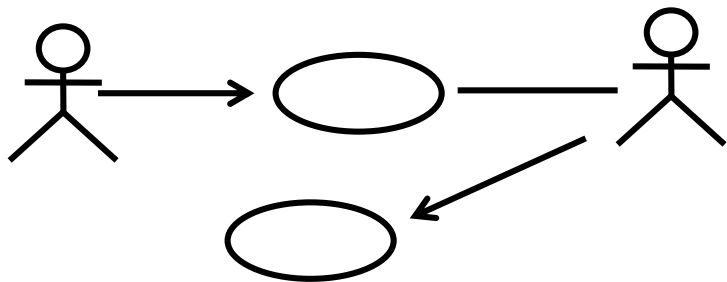
Part 03-3

Requirements Analysis

Use Case Specification

Use Cases **Contain** Software Requirements

- Each Use Case
 - Describes actions the system takes to deliver something of value to the actor
 - Shows the system functionality an actor uses
 - Models a dialog between the system and actors
 - Shows **a complete and meaningful flow of events from the perspective of a particular actor**



Detail a Use Case

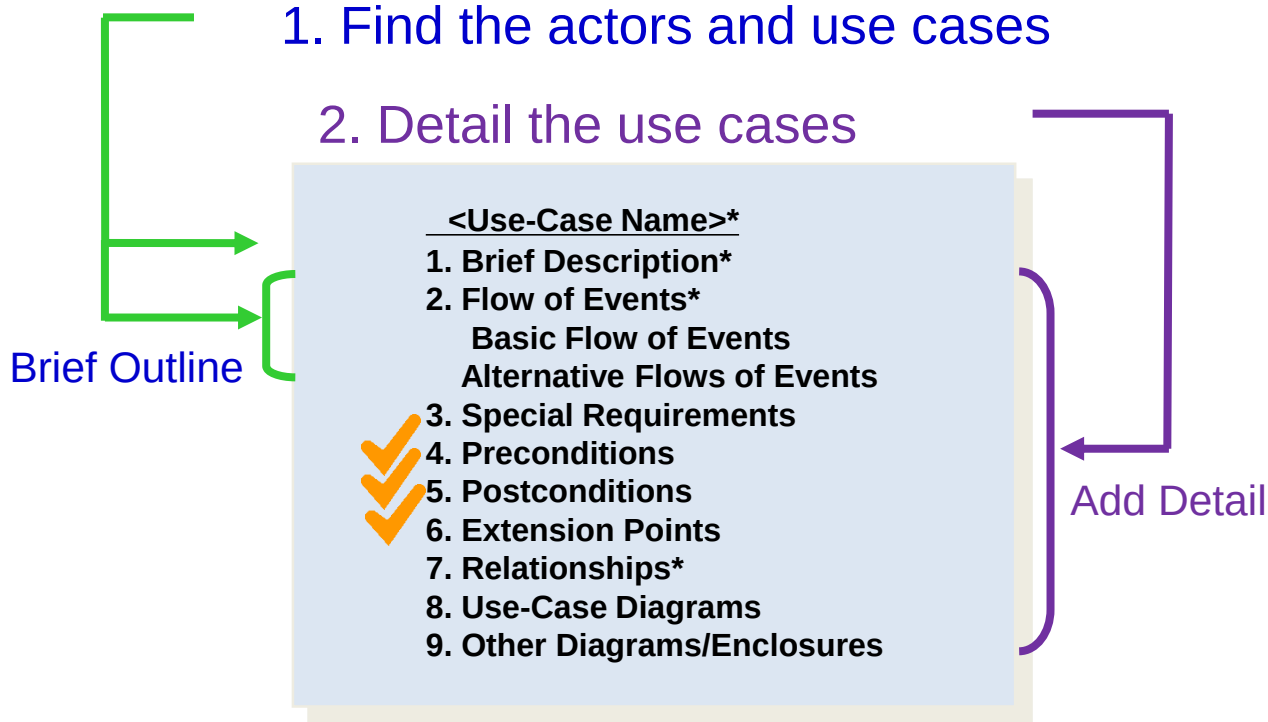
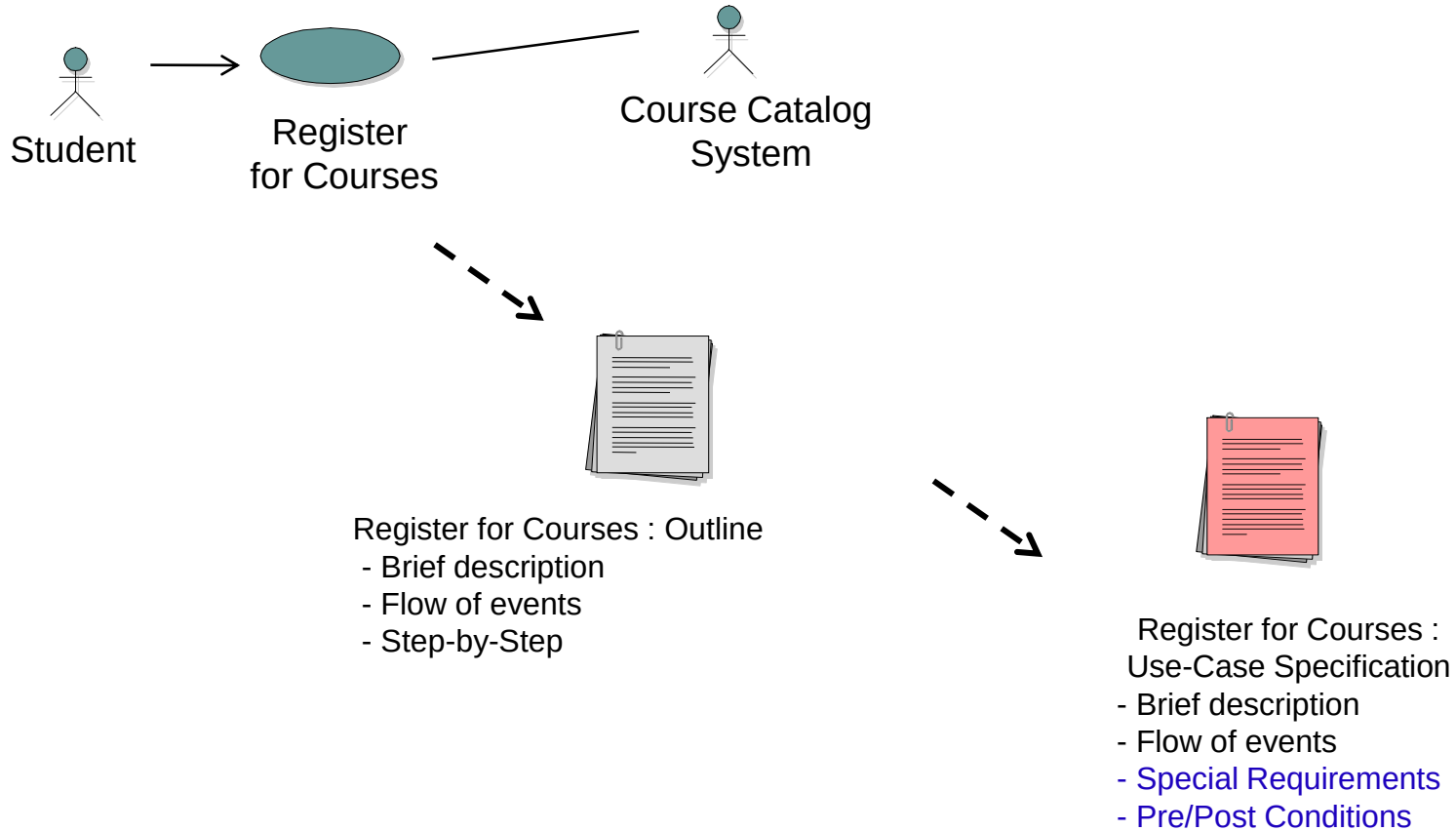


Diagram → Outline → Detail



Use Case Outline: 수강신청

Use Case Name	수강신청
Brief Description	학생들이 현재 학기의 수강신청을 하는 유스케이스. 최초 수강신청은 물론, 수강정정 기간 중에 기존 수강신청 내용을 변경하는 것도 가능함
Principal Actor	학생, 과목정보시스템(legacy)
Precondition	-
Basic Flow	1. 수강신청/정정/취소 선택 - 최초 수강신청인 경우: 수강신청(step2) 서브플로우 수행 - 기존 수강신청 과목변경의 경우, 수강정정(step3) 서브플로우 수행 - 기존 수강신청 취소, 수강신청 취소(step4) 서브플로우 수행 2. 수강신청 2.1 개설강좌 목록 조회 2.2 수강과목 선택 2.3 해당학기 수강신청 스케줄 생성 2.4 수강신청 스케줄 제출 완료 2.5 수강신청 확정 3. 수강정정 3.1 현재 수강신청 리스트업 3.2 추가 가능 과목정보 리스트업 3.3 수강취소 과목 선택 및 대체 과목 선택 3.4 수정된 수강신청 스케줄 저장 3.5 수강정정 확정 4. 수강취소 4.1 현재 수강신청 리스트업 4.2 수강신청 취소 4.3 수강신청 취소 확인(confirm)

Use Case Outline: 수강신청

Alternative Flow

- AF1. 수강신청 실패
- AF2. 저장된 수강신청 스케줄이 없음
- AF3. 과목정보 시스템 접근 불가능
- AF4. 수강신청 기간 종료
- AF5. 수강취소 철회

Detail the Basic Flow of Events

Select courses to teach

Basic Flow

The use case starts when a Professor wishes to sign up to teach some course offerings for the upcoming semester.

1. Retrieve and display the list of course offerings

The system retrieves and displays the list of course offerings the professor is eligible to teach for the current semester. The system also retrieves and displays the list of courses the professor has previously selected to teach.

2. Professor selects course offerings

The professor selects and/or de-selects the course offerings that he/she wishes to teach for the upcoming semester.

3. Remove the professor

The system removes the professor from teaching the de-selected course offerings....

Structure the flow into steps

Number and title each step

Describe steps (1-3 sentences)

Make each step a roundtrip of events

Detail Specific Alternative Flows

Register for courses

Alternative Flows

A1. No Schedule Found

If, in the **Submit Schedule** sub-flow, the system determines that the Student has not satisfied the necessary prerequisites, or that the selected course offering is full, or that there are schedule conflicts, an error message is displayed. The Student can either select a different course offering and the use case continues, save the schedule, as is (see **Save a Schedule** subflow), or cancel the operation, at which point the **Basic Flow** is re-started at the beginning.

Describe what happens

Start

Condition

Actions

Resume

Alternative Flow: Example

- **A5. 환율 엑셀파일 Loading선택의 경우 → Condition**

- 분기점 : 대체흐름(A1) 7 → **Start**

1. 시스템은 Telerate, Reuter로부터 받은 환율자료에 대한 전환자료가 존재하는 해당위치의 엑셀파일을 Loading해서 화면의 각 항목에 display한다.
2. 엑셀파일 자동Loading 실패인 경우, E4수행
3. 본점Teller는 화면의 내용을 검토한 후 수정할 내용에 대한 수정작업을 수행한 후 저장버튼을 선택한다.
4. 대체흐름(A1) 8로 돌아감 → **Resume**

Use Case Specification: 수강신청

Use Case Name	수강신청
Brief Description	학생들이 현재 학기의 수강신청을 하는 유스케이스. 최초 수강신청은 물론, 수강정정 기간 중에 기존 수강신청 내용을 변경하는 것도 가능함
Principal Actor	학생, 과목정보시스템(legacy)
Precondition	학생은 이미 시스템에 로그인된 상태(로그온된 상태여야 한다는 내용은 생략해도 무관, 특별한 Precondition이 필요한 경우만 기술함)
Basic Flow	
본 유스케이스는 학생이 자신의 수강과목을 신청하거나, 신청된 수강신청을 정정하거나 취소하기 위해 수강신청 어플리케이션의 해당 메뉴를 선택함으로써 시작된다.	
1. 수강신청/정정 선택	
- 최초 수강신청인 경우: 수강신청 (step 2) 서브플로우 수행	
- 기존 수강신청 과목변경의 경우, 수강정정 (step 3) 서브플로우 수행	
- 기존 수강신청 취소, 수강신청 취소 (step 4) 서브플로우 수행	
2. 수강신청	
2.1 개설강좌 목록 조회	
시스템은 과목정보시스템으로부터 현재 개설된 과목 각각의 과목코드, 과목명, 선수과목코드, 선수과목명칭, 학점수, 강의시간, 담당교수명 등의 정보를 읽어와서 (AF3), 수강신청 화면의 개설과목 목록란에 리스트업한다. 이때, 담당 교수가 아직 할당되지 않은 경우에는 해당 항목은 공란인 상태로 표시한다.	
2.2 수강과목 선택	
학생은 개설과목 목록에 출력된 과목 중, 자신이 수강하고자 하는 과목을 선택한 후 선택 버튼을 클릭한다. 시스템은 선택된 과목에 해당되는 행을 수강신청 화면의 개설과목 목록 아래에 있는 수강신청 과목 목록으로 이동시킨다.	
학생이 한 학기에 수강 신청할 수 있는 과목은 총 4과목이다. 수강인원 초과 등의 이유로 수강이 불가능한 경우를 대비하여 2개의 과목을 추가적으로 더 선택한다.	
2.3 해당학기 수강신청 스케줄 생성	
학생은 6과목을 모두 선택한 후, 수강신청 화면 하단의 저장 버튼을 선택한다. 시스템은 수강신청한 학생의 학번, 이름, 현재학사년도, 현재 학기, 그리고 선택한 6개의 과목 관련 정보(과목코드, 과목명, 선수과목코드, 선수과목명칭, 학점수, 강의시간, 담당교수명)를 포함하는 수강신청 스케줄을 시스템 상에 임시 저장한 후, "저장되었습니다" 라는 메시지를 화면상에 출력한다.	

Use Case Specification: 수강신청

Basic Flow

2.4 수강신청 스케줄 제출 완료

학생은 저장된 수강신청 스케줄을 검토한다. 필요한 선수과목을 모두 이수하였고 수강 신청하는 과목의 강의시간 중 겹치는 과목이 없는지 최종 확인한 후, 문제가 없을 경우, 화면 하단의 수강신청완료 버튼을 선택한다. 시스템은 생성된 수강신청의 상태를 '제출'로 변경한 후 "수강신청이 제출되었습니다" 라는 메시지를 화면상에 출력한다.

2.5 수강신청 확정

시스템은 "제출"된 수강신청 스케줄에 포함된 모든 과목의 선수과목을 학생이 이미 이수한 상태인지, 과목의 강의 시간들 간의 충돌은 없는지, 이미 수강신청 인원이 초과된 과목이 없는지 체크 (AF1) 한 후 문제가 없을 경우 수강신청 스케줄의 상태를 "등록"으로 변경한 후 본 유스케이스를 종료한다.

3 수강정정

3.1 현재 수강신청 리스트업

시스템은 현재시점이 수강정정 기간인지 확인 (AF4) 한 후, 해당학생의 수강신청 정보(과목코드, 과목명, 선수과목코드, 선수과목명칭, 학점수, 강의시간, 담당교수명)를 수강정정 화면에 리스트업한다. (AF2). 이때 화면의 상단에는 해당 학생의 학번 이름, 학사년도, 학기를 함께 디스플레이한다.

3.2 추가 가능 과목정보 리스트업

시스템은 현재 개설된 과목들의 정보도 화면 하단의 목록에 함께 리스트업하여 보여준다.

3.3 수강취소 과목 선택 및 대체 과목 선택

학생은 수강정정 화면의 수강신청과목 목록에서 자신이 drop하고자 하는 과목을 선택한 후, 목록 하단의 과목취소 버튼을 선택한다. 시스템은 해당 과목을 학생의 수강신청 스케줄에서 삭제한다.

학생은 취소한 과목 대신 수강할 과목을 개설과목 목록에서 선택한 후, 목록 하단의 추가 버튼을 선택한다. 시스템은 선택된 과목행을 화면 상단의 수강신청 과목 목록으로 이동시킨다.

3.4 수정된 수강신청 스케줄 저장

학생은 수정된 수강신청 스케줄을 검토한다. 필요한 선수과목을 모두 이수하였고 수강 신청하는 과목의 강의시간 중 겹치는 과목이 없는지 최종 확인한 후, 문제가 없을 경우, 화면 하단의 수강신청완료 버튼을 선택한다. 시스템은 생성된 수강신청의 상태를 "수정"으로 변경한 후 "변경된 수강신청이 저장되었습니다" 라는 메시지를 화면상에 출력한다.

3.5 수강정정 확정

시스템은 "수정"된 수강신청 스케줄에 포함된 모든 과목의 선수과목을 학생이 이미 이수한 상태인지, 과목의 강의 시간들 간의 충돌은 없는지, 이미 수강신청 인원이 초과된 과목이 없는지 체크한 후 문제가 없을 경우 수강신청 스케줄의 상태를 "수정등록"으로 변경한 후 본 유스케이스를 종료한다.

Use Case Specification: 수강신청

Basic Flow

4. 수강취소

4.1 현재 수강신청 리스트업

시스템은 현재시점이 수강정정 기간인지 확인한 후, 해당학생의 수강신청 정보(과목코드, 과목명, 선수과목코드, 선수과목명칭, 학점수, 강의시간, 담당교수명)를 수강정정 화면에 리스트업한다. 이때 화면의 상단에는 해당 학생의 학번, 이름, 학사년도, 학기를 함께 디스플레이한다.

4.2 수강신청 취소

학생은 수강신청과목 목록 하단의 수강신청취소 버튼을 클릭한다. **시스템**은 화면상에 '현재 수강신청을 정말 취소하시겠습니까?'라는 메시지를 출력한다.

4.3 수강신청 취소 확인(confirm)

학생이 취소확인 버튼을 선택 (AF5) 하면 **시스템**은 해당 학생의 해당학기 수강신청 정보를 모두 삭제한 후 **본 유스케이스를 종료한다**.

Use Case Outline: 수강신청

Alternative Flow

AF1. 수강신청 실패

분기점: Basic Flow 2.5

시스템은 수강신청 실패 이유에 따라 "수강신청 과목 중 강의 시간이 중복되는 과목이 둘 이상 존재합니다.", "***과목을 수강하기 위해서는 ***과목을 이수하여야 합니다.", "***과목의 수강 인원이 초과되어 수강신청이 불가합니다" 등과 같은 적절한 메시지를 화면상에 출력한 후, 본 유스케이스를 종료한다.

AF2. 저장된 수강신청 스케줄이 없음

분기점: Basic Flow 3.1

시스템은 "조회할 수강신청 데이터가 저장되어 있지 않습니다."라는 메시지를 화면상에 출력한 후, 본 유스케이스를 종료한다.

AF3. 과목정보 시스템 접근 불가능

분기점: Basic Flow 2.1

시스템은 "현재 과목정보 시스템에 접근이 되지 않습니다."라는 메시지를 화면상에 출력한 후, 본 유스케이스를 종료한다.

AF4. 수강정정 기간이 아닐 경우

분기점: Basic Flow 3.1

시스템은 "수강정정 기간이 아닙니다."라는 메시지를 화면상에 출력한 후, 본 유스케이스를 종료한다.

AF5. 수강취소 철회

분기점: Basic Flow 4.3

학생이 취소철회 버튼을 클릭한 경우, 시스템은 현재의 수강신청 정보를 그대로 두고, 본 유스케이스를 종료한다.



Part 03-4

Requirements Analysis

Quality Attribute Scenarios

The State-of-the-Art Requirements Modeling Methods

1. Structured analysis

- Data Flow Diagram (DFD)
- State Transition Diagram (STD)
- Entity-Relation Diagram (ERD)

2. Use Case analysis

- Use Case Modeling (UC)

3. Architectural trade-off analysis

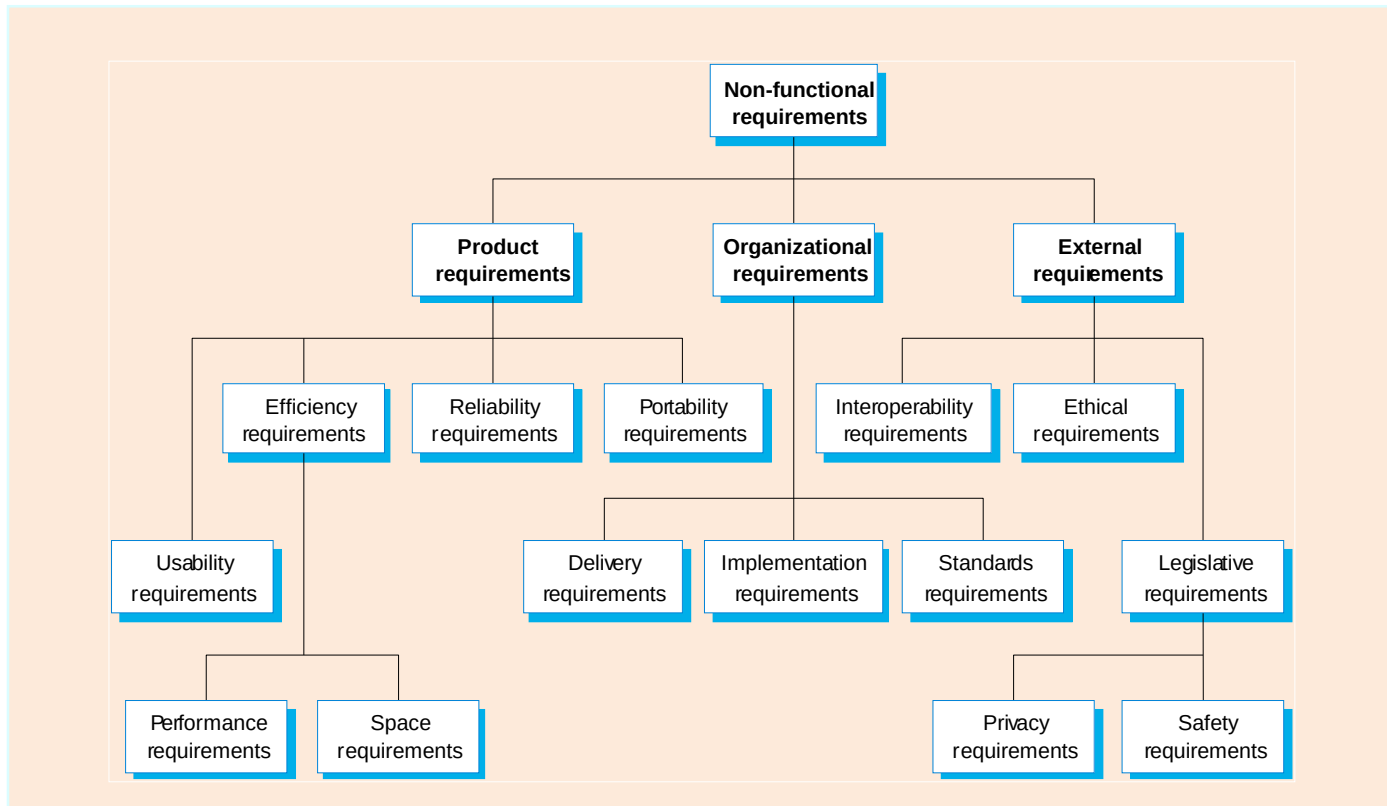
- Quality Attribute Scenario (QAS)

Scenario-based Requirements Model

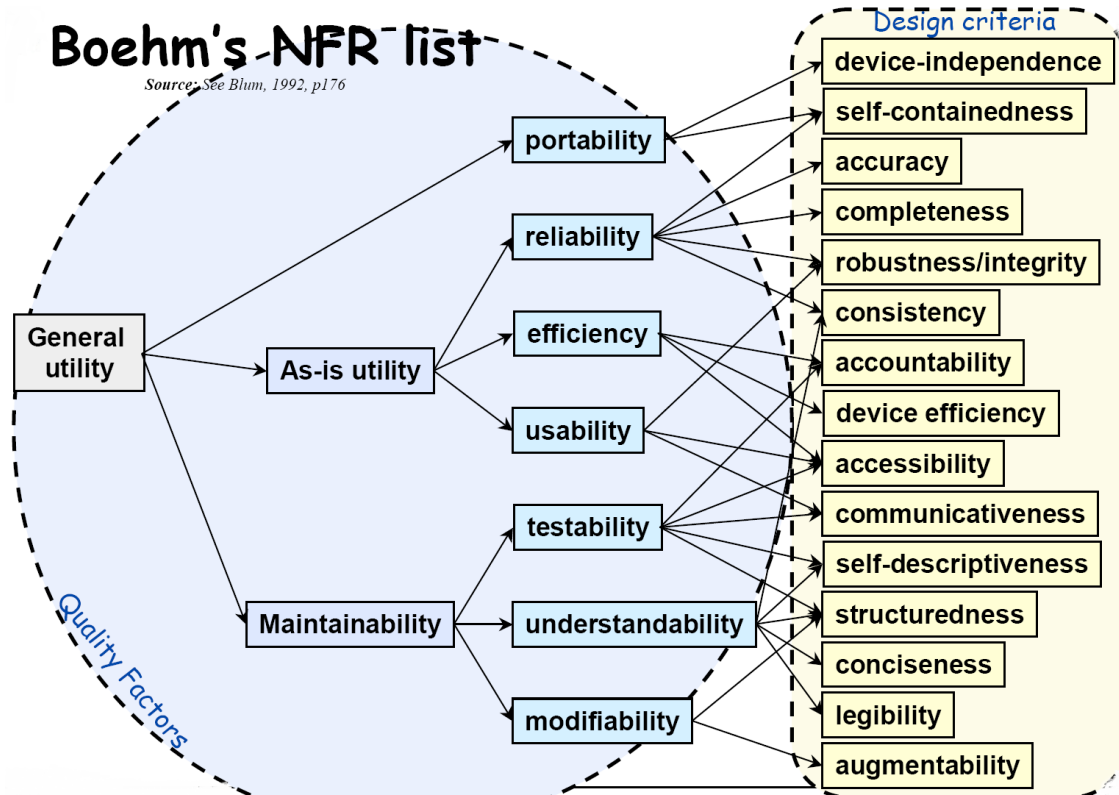
Non-Functional Requirements

4. Goal Analysis

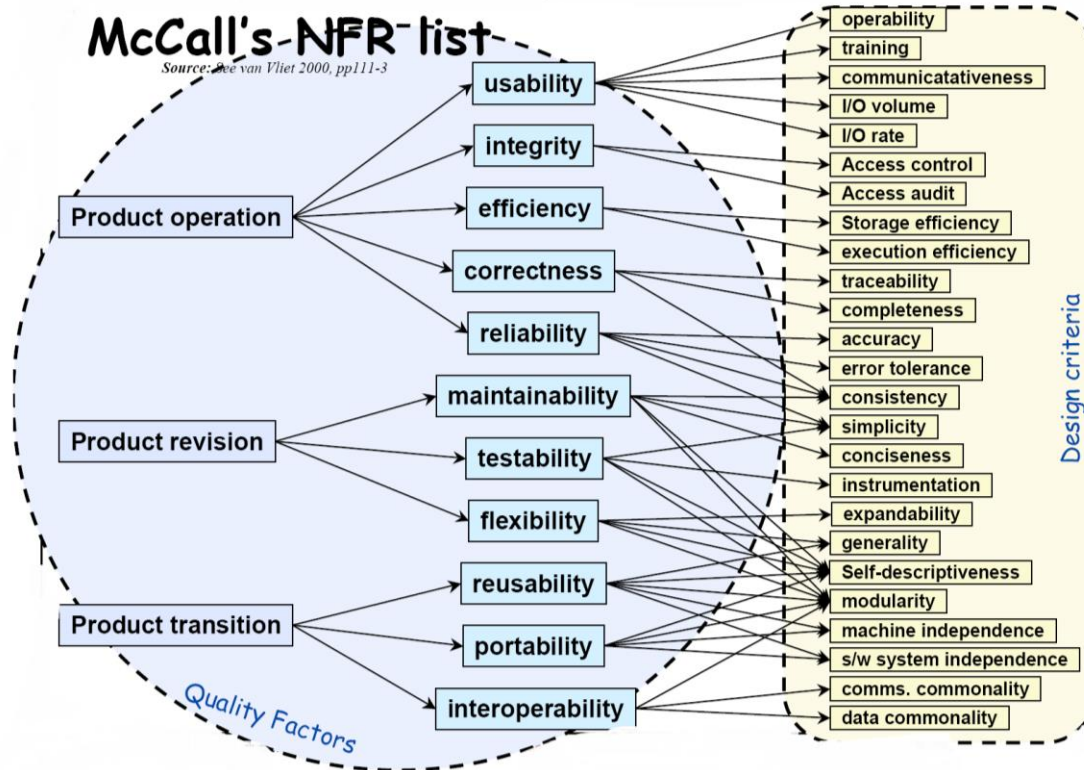
Quality Attributes Taxonomy Examples



Quality Attributes Taxonomy Examples



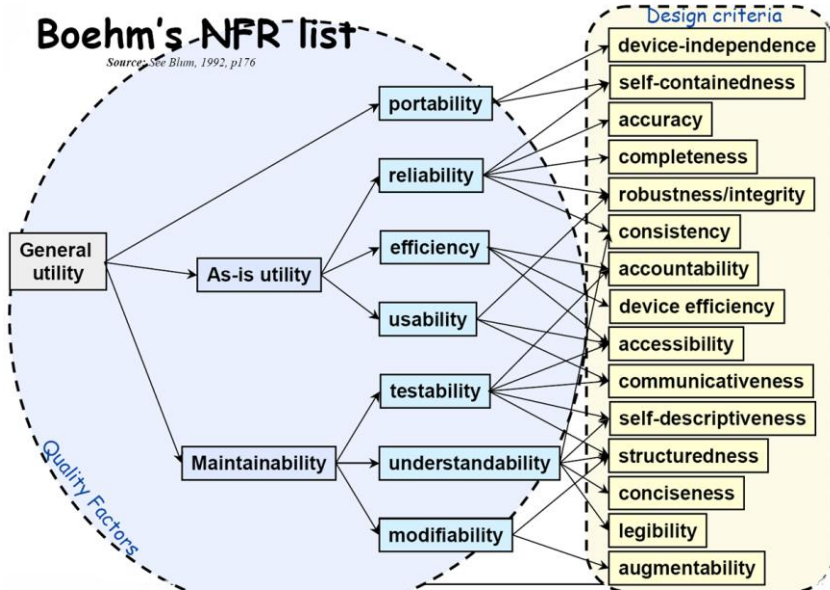
Quality Attributes Taxonomy Examples



Quality Attributes Taxonomy Examples

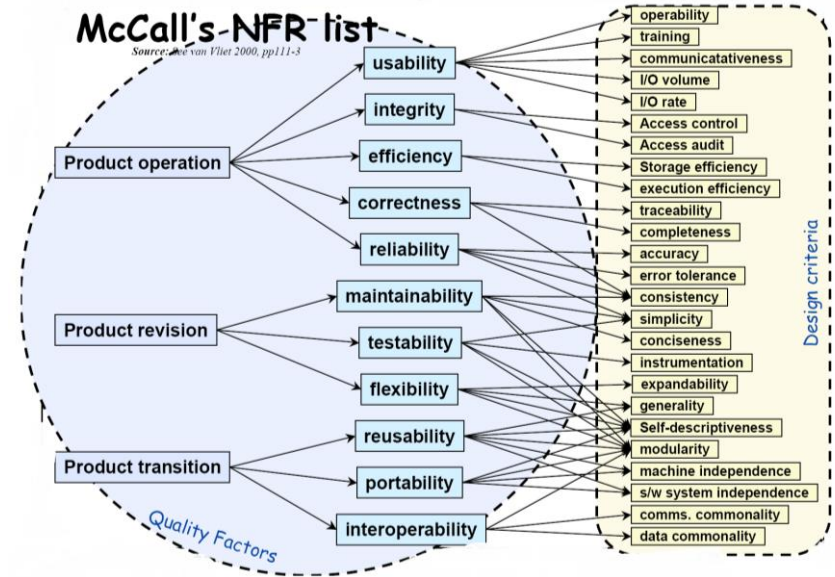
Boehm's NFR list

Source: Sie Blum, 1992, p176



McCall's NFR list

Source: Joe von Vliet 2000, pp111-3



Quality Attributes Taxonomy Examples: FURPS+

- Classification system devised by Robert Grady at Hewlett-Packard

- **FURPS**



Architecturally Significant → 어느 하나의 function에만 적용되는 것이 아니라, 다양한 function들을 cross-cutting 하여 적용되어야 하는 기능들

- **F**unctionality
 - Main product feature
 - **Architecturally Significant Functional Requirements:** Auditing, Licensing, Localization, Mail, Online help, Security, System management, Printing, etc.
- **U**sability
- **R**eliability
- **P**erformance
- **S**upportability: testability, adaptability, maintainability, compatibility, configurability, installability, scalability, and localizability
- **The "+" in FURPS+ also helps us to remember concerns such as:**
 - Design requirements
 - Implementation requirements
 - Interface requirements
 - Physical requirements(HW requirements)

알쏭달쏭한 Requirements Category

- **Non-Functional Requirements 와 Quality Attribute는 같은 개념?**

- 같은 개념이다...
- QA는 NFR의 Subset이다...

→ Constraints를 NFR의 범주 아래에 두는지, 혹은 function을 cross-cutting하는 common system feature를 무엇으로 보는지에 따라 두 용어가 지칭하는 개념 간의 coverage는 달라질 수 있음

- **Reliability와 Availability는 어떻게 다른 Quality Attribute들인가?**

- "System이 믿을 만 하다." 는 것과 "System이 가용성이 높다"의 차이는??
- 이렇게 구분하는 사람도 있는 반면,
 - Availability is a measure of the percentage of time that an IT service or component is in an operable state.
 - Reliability, on the other hand, is a measure of the probability that the system will meet defined performance standards in performing its intended function during a specified interval (MTBF : Mean Time Between Failure)
- CMU SEI에서의 Quality Attribute 구분에서는 위의 두 가지를 뭉뚱그려 Availability 속성으로 정의하고 있음

Requirements taxonomy는 결국, 모든 Requirements를 빠짐없이 식별하고자 만든 것
→ 서로 다른 taxonomy 체계가 왜 다른 지에 집착할 필요는 없음

Quality Attributes

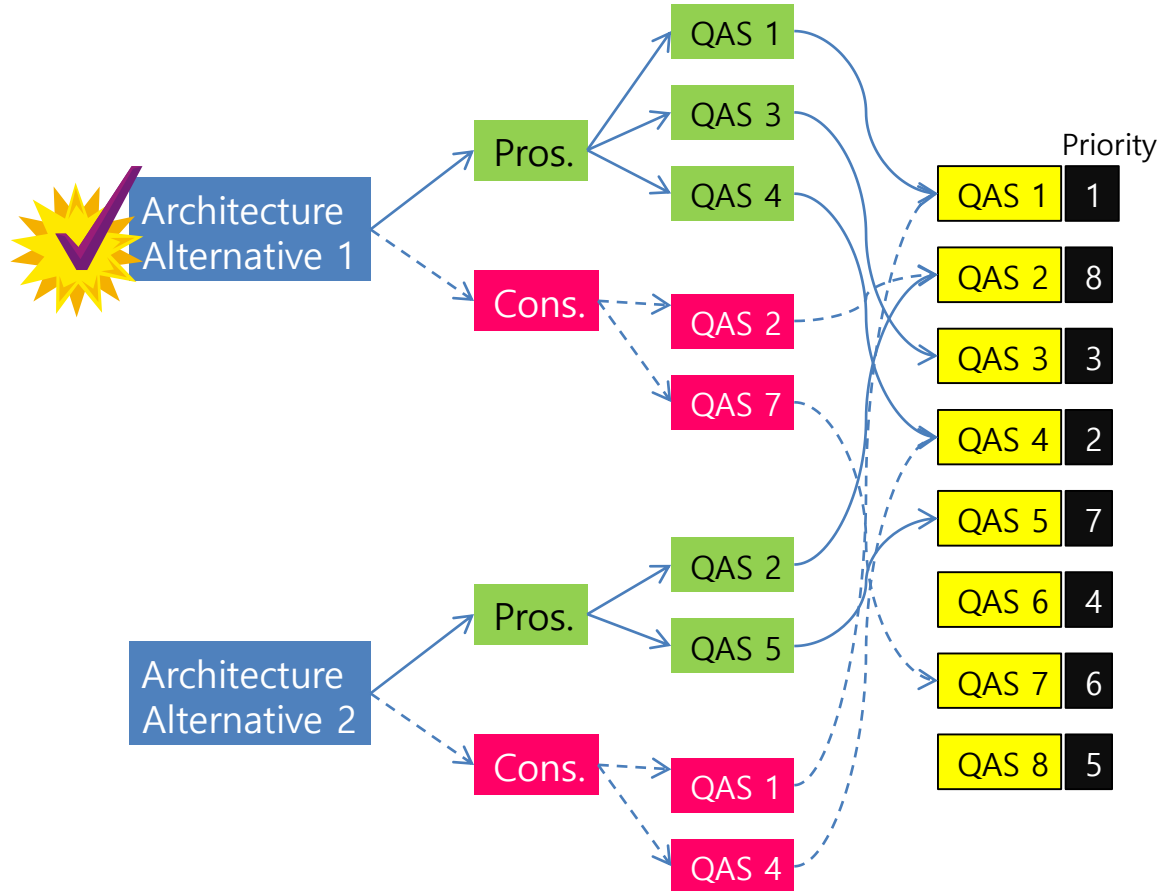
- **Measurable or testable properties of a system**

- Used to indicate how well the system satisfies the needs of its stakeholders
 - Availability, configurability, modifiability, performance, reliability, reusability, security, portability, maintainability, efficiency, usability
- Emergent properties : not a measure of software in isolation
 - Measures the relationship between software and its application domain
 - Cannot measure this until you place the software into its environment
 - Quality will be different in different environments

소프트웨어 자체만을 보고 quality를 평가하는 것이 아니라 소프트웨어와 그것이 적용되는 도메인 간의 관계를 measure하는 새로운 평가 방식

- Software quality is all about **fitness to purpose** of stakeholders.
 - "Does it do what is needed?"
 - "Does it do it in the way that its users need it to?"
 - "Does it do it reliably enough? fast enough? safely enough? securely enough?"
 - "Will it be affordable? will it be ready when its users need it?"
 - "Can it be changed as the needs change?"

Analyze the Architectural Approaches

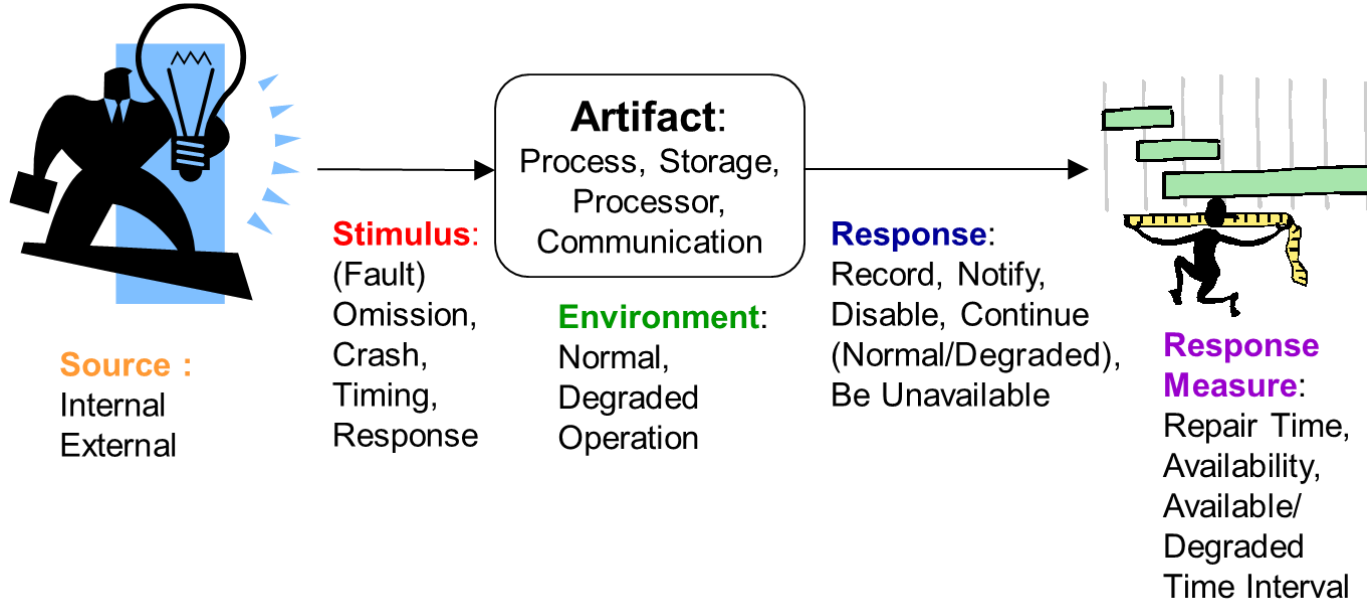


Quality Attribute Scenarios

- **QAS (Quality Attribute Scenario)**

is an effective way of identifying and specifying quality-attribute-specific requirements.

- Specific to the particular system under considerations
- Instantiated from the attribute characterizations of general scenarios



CMU-SEI가 제시하는 QAS를 활용하여 QA명세하기

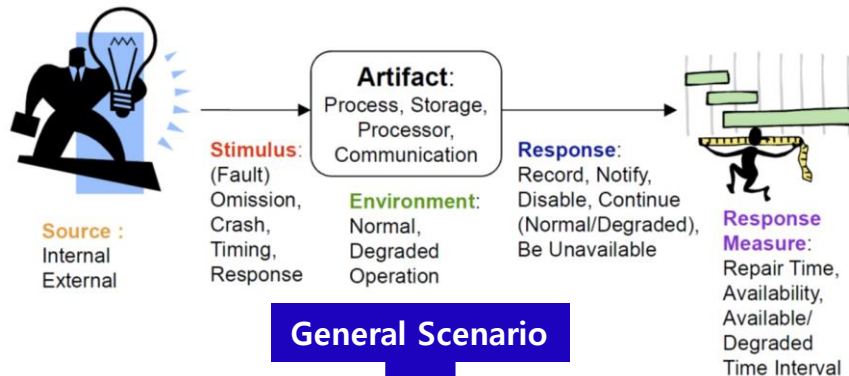
1. 각각의 Quality Attribute 속성을 이해
2. 해당 Quality Attribute의 General QAS의 각 항목에 활용 가능한 Possible Values를 파악
3. Possible Value들을 QAS Template의 각 항목에 배치하여, 개발 대상인 Software System의 Quality Attribute를 규정(Scenario Instantiation)

Step 1. 각각의 Quality Attribute 속성을 이해

- Quality Attribute - Availability
 - Ability of a system to mask or repair faults such that the cumulative service outage period does not exceed a required value over a specified time interval
 - The availability of a system can be calculated as the probability that it will provide the specified services within required bounds over a specified time interval.
 - When referring to hardware, there is a well-known expression used to derive steady-state availability:
 - MTBF : the mean time between failures
 - MTTR : the mean time to repair

Step 2. 해당 Quality Attribute의 General QAS의 각 항목에 활용 가능한 Possible Values를 파악

• Quality Attribute Scenario - Availability



General Scenario

Scenario instantiation

“An **unanticipated external message** is received by a process during **normal operation**. The process **informs the operator of the receipt of the message** and **continues to operated with no downtime**.”

A Concrete Scenario

Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	Fault: omission, crash, incorrect timing, incorrect response
Artifact	Processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: - Log the fault - Notify appropriate entities (people or systems) Recover from the fault: - Disable source of events causing the fault - Be temporarily unavailable while repair is being effected - Fix or mask the fault/failure or contain the damage it causes - Operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g., 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

QAS Template

Requirement-ID	Quality Attribute 명칭_##
Category	관련된 Quality Attribute가 무엇인지(ex Performance, Reliability, Security 등등)
Source	Stimulus를 발생시키는 주체가 무엇인지 기술
Stimulus	시스템에 입력되는 내외부 자극이 무엇인지 기술
Environment	해당 Stimulus 발생시 시스템의 환경(운영모드일 수도 있고, 그 외 다양한 상황 가능)
Artifacts	Stimulus의 영향을 받는 시스템의 내부 모듈, 컴포넌트, 혹은 시스템 전체
Response	기술된 Environment에서 Artifact이 Stimulus를 받아들인 후 취하는 Action이 무엇인지
Response Measure	위의 Response의 정도를 측정하는 단위가 무엇인지(ex 초당 데이터 처리량, 반응시간(시간일 경우 Hr인지, Min인지, Sec인지 등등))
Priority	Quality Attribute Tree 상에서의 우선순위
Description	위에 기술된 Source부터 Response Measure까지의 내용을 하나의 문장으로 요약해서 기술. 본 항목이 실제 Quality Attribute Scenario에 해당되며, Quality Attribute Tree의 Leaf Node에 나열됨

Step 3. Scenario Instantiation

Portion of Scenario	Possible Values
Source	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	Fault: omission, crash, incorrect timing, incorrect response
Artifact	Processors, communication channels, persistent storage, processes
Environment	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	Prevent the fault from becoming a failure Detect the fault: - Log the fault - Notify appropriate entities (people or systems) Recover from the fault: - Disable source of events causing the fault - Be temporarily unavailable while repair is being effected - Fix or mask the fault/failure or contain the damage it causes - Operate in a degraded mode while repair is being effected
Response Measure	Time or time interval when the system must be available Availability percentage (e.g., 99.999%) Time to detect the fault Time to repair the fault Time or time interval in which system can be in degraded mode Proportion (e.g., 99%) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing



Requirement-ID	QA_01
Category	Availability
Source	external
Stimulus	An unanticipated message is recieved
Environment	During normal operation
Artifacts	A process
Response	Informs the operator of the receipt of the message and continues to operate
Response Measure	With no downtime
Priority	3

A competed description as a QAS for Availability

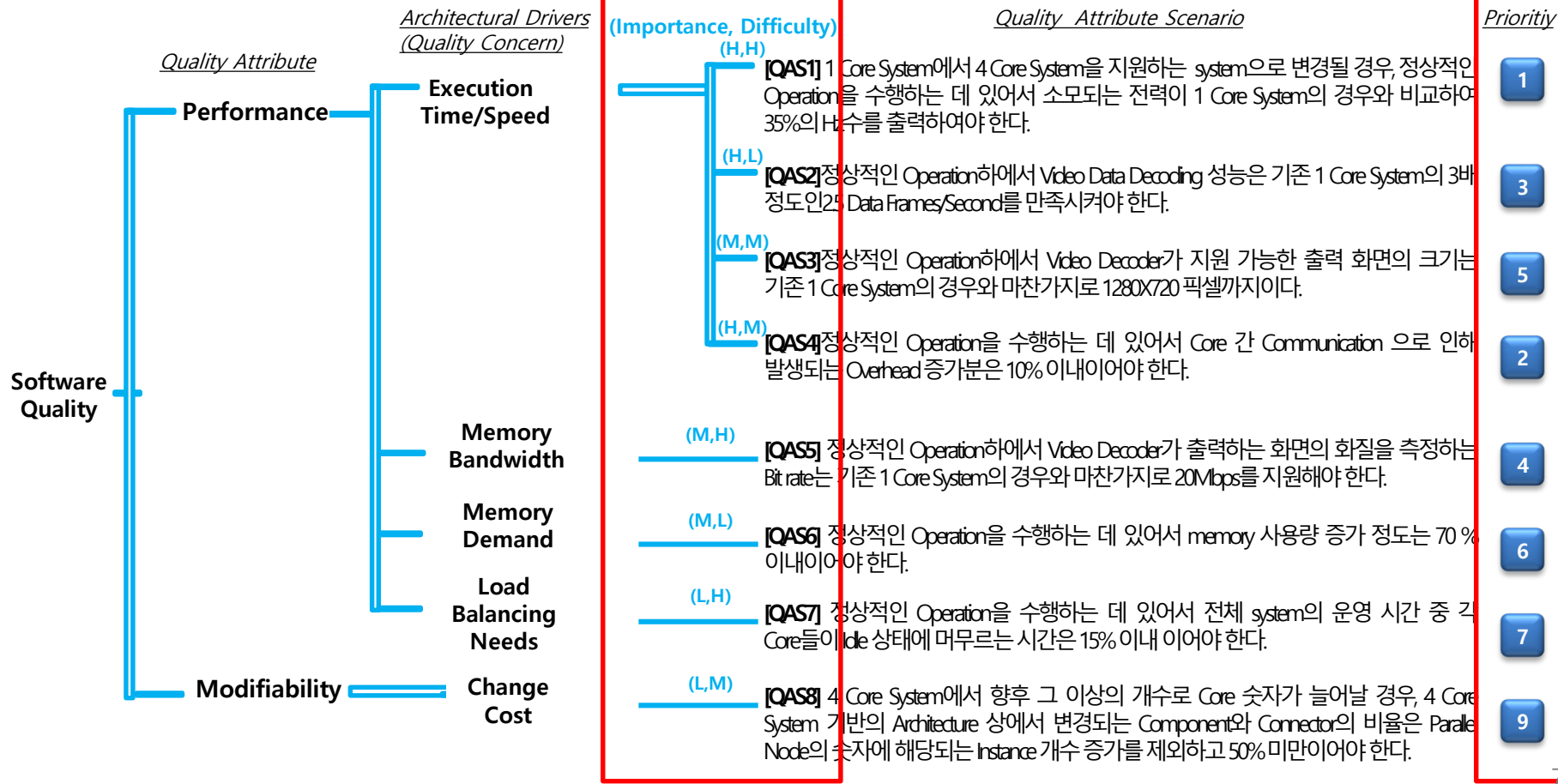
"An **unanticipated external message** is received by a process during **normal operation**. The process **informs the operator of the receipt of the message and continues to operated with no downtime**."

Quality Attribute Scenario의 우선순위 결정시 고려요소

- Importance: business stakeholder가 부여하는 상대적인 Business Value 측면의 중요도
- Difficulty: technical stakeholder가 부여하는 대상 QAS를 성취하는데 요구되는 상대적인 effort
- (I,D) 조합에 따른 우선순위:
(H, H) > (H, M) > (H, L) > (M, H) > (M, M) > (M, L) > (L, H) > (L, M) > (L, L) 순으로 높은 순위를 가짐

I. Ozkaya, L. Bass, R. S. Sangwan and R. L. Nord, "Making Practical Use of Quality Attribute Information," in *IEEE Software*, vol. 25, no. 2, pp. 25-33, March-April 2008, doi: 10.1109/MS.2008.39.

An Example of Quality Attribute Tree



Exercise 1. 사례의 Non-Functional Requirements

- 음료수 자판기 컨트롤러의 Quality Attribute로는 어떤 속성이 정의되어야 할까요?
 - Q1. 음료수 자판기 컨트롤러의 고객은 누구인가요? 음료수를 사는 고객? 자판기 주인?
자판기 주인
 - Q2. 그렇다면 자판기 주인이 요구하는 음료수 자판기 컨트롤러(혹은 자판기 자체)의 Quality Attribute는 무엇??
 1. Availability: 항상 음료수는 판매 가능한 상태인가?
 2. Security: 자판기 주인 이외의 사람의 관리자 모드 접근을 얼마나 잘 방지하는가?
 3. Performance: 자판기 시스템은 판매하는 음료의 적정 온도를 얼마나 잘 유지시켜 주는가?

QAS Example: 로봇청소기 QAS

작성한 QAS	Comment
로봇청소기가 정상적으로 청소를 수행하는 도중, 사용자의 리모컨 “빠른청소기능” 신호를 받게 되면, 1초(measure) 이내로 “빠른청소기능”이 수행된다.	1
바닥의 먼지를 흡입하는 상태에서 ‘중간’ 세기의 흡입강도를 설정해 놓은 경우 직경 1mm 이상, 100mm 이하의 먼지 알갱이에 대해 흡입동작 이후 잔류한 먼지 알갱이의 개수가 시험을 위해 살포한 전체 먼지 알갱이 개수의 0.01 % 이하여야 한다.	
충전기를 벗어나면 충전기까지 돌아가는 데 필요한 전력을 계속 계산하고 있어야 한다. 청소 중 충전기까지 바로 가도 배터리 잔량이 2% 이하가 될 것이 예상되면 청소를 중단하고 충전기로 돌아간다 현재 알고 있는 경로로 충전기로 돌아가던 중 장애물을 만날 시 좌/우 더 넓은 공간으로 우회한다. 우회해도 경로는 찾지 못하는 경우 알람을 울려 이상 상태를 표시한다.	2
사용자가 정해놓은 청소주기가 도래하면 대기 상태인 로봇청소기의 휠 구동부는 1초이내 3km/h까지 가속을 완료해야 한다.	2
사용자가 대기 상태의 로봇 청소기에게 리모컨으로 청소 요청을 명령했을 시, 로봇 청소기는 2초 이내에 청소 시작하여야 한다.	1
로봇청소기가 청소 모드 또는 충전을 위한 이동 중 이동 경로상에 사용자 이동시 30ms안에 이를 감지 후 사용자와 최소 30cm 떨어진 새로운 경로로 변경하여야 한다. 사용자가 로봇청소기와 30cm 이하에 접근 시 로봇청소기는 동작을 멈추어야 한다.	2
정상 운영 모드로 자동 청소 수행 시 다음 충전 전에 100m ³ 의 공간을 청소하여야 한다.	
먼지가 많은 사용 환경에서 먼지가 거리감지 센서를 오염시켰을 경우에도 거리를 정상적으로 인지하여 장애물을 회피해야 함	3
사용자가 대기 중인 로봇청소기의 자동 청소 버튼을 누르면 방 전체의 95% 이상 청소 되었을 때 청소 완료 표시 등이 켜져야 한다.	2
청소기 버튼이 눌렸을 때 전원이 인가된 상태에서 모터가 동작하여 먼지를 빨아들이는 동작을 0.5초 이내로 수행되어야 한다	1
사용자가 완충 및 전원이 Off 된 상태의 로봇 청소기의 전원 버튼을 동작시켜 전원을 On 했을 경우, 로봇 청소기는 10 초 이내에 내부 스피커를 통한 소리를 발생시켜 청소 준비가 되었음을 알린다.	2
일반 가정집 환경에서 사용자가 로봇 청소기의 표준 자동 청소 기능을 동작시킬 경우, 자동 청소 종료 시 가용 청소 영역의 95% 이상이 청소되어야 한다.	3
사용자가 청소 작업을 시작하도록 하는 경우, 표준 거실 환경에서 청소 대상 모두를 청소하고 30분 내에 제자리로 돌아와야 한다.	
청소하고 난 후의 구역은 단위면적당 먼지 양이 최대 1g/m ² 을 넘겨서는 안 된다.	
사용자가 일반 가정집에서 로봇청소기를 (얼마동안???)가동시키면 95% 이상의 면적이 청소 되어야 한다.	3
Max 모드에서 20m x 20m 공간을 청소하는 동안 청소기소음이 100dB를 넘지 않는다.	
정상모드에서 사용자가 로봇청소기에 흡입 세기를 변경한 경우 평균 0.3초 시간 안에 변경한다.	1
사용자가 일반적인 상황에서 자동 청소 기능을 시작 했을 경우 로봇 청소기는 10분안에 전체 청소 대상 영역(대상영역 넓이 명시 필요)에 대해 청소를 완료 하여야 한다	3
로봇청소기 이동 구간에 장애물이 있는 경우 1초 이내에 장애물을 인식하고 이동방향을 변경	2
사용자가 로봇 청소기의 전원을 켜다. 로봇 청소기는 전원 입력 후 30초 이내에 동작 대기 상태로 전환 한다.	1
사용자(source) 로봇청소기 시스템의 전원을 켜고(environment) 청소할 범위를 입력(stimulus)하면 구역청소컨트롤러(artifacts)는 1m2 당 최대 1분의 시간(response measure)을 소요하여 구역을 탐색&청소한다. (response)	
Normal 청소 모드 중 먼지 입자의 농도가 30ppm 이상인 경우 모터의 가동력을 500ms이내에 normal mode의 120%로 증가 시킨다.	2
시스템 가동 상태에서 청소 시작이 입력되었을 때 10mx10m의 사이즈의 거실 형태(tv장,소파, 서랍장이 있는)로 배치된 방의 청소를 5분내에 완료하여야 한다.	

로봇청소기 QAS 작성사례에서의 문제점_1

- 로봇청소기가 정상적으로 청소를 수행하는 도중, 사용자의 리모컨 “빠른청소기능” 신호를 받게 되면, 1초(measure) 이내로 “빠른청소기능”이 수행된다.
- 사용자가 대기 상태의 로봇 청소기에게 리모컨으로 청소 요청을 명령했을 시, 로봇 청소기는 2초 이내에 청소 시작하여야 한다.
- 청소기 버튼이 눌렸을 때 전원이 인가된 상태에서 모터가 동작하여 먼지를 빨아들이는 동작을 0.5초 이내로 수행되어야 한다

• 로봇청소기의 구입을 결정할 만한 품질요소를 정의하고 있는가?

- 로봇이 아닌 일반 청소기의 성능요소(흡입력, 소음, 배출구로 배출되는 먼지의 양, 배터리 용량) + 로봇청소기의 성능요소(단위시간당 청소작업의 coverage, 청소 후 잔여먼지 농도 등) 등이 품질 요소로 적합 → But, 이 중에서 SW의 quality facto가 어느 정도인지가...
- 버튼을 눌렀을 때 반응속도/모드 전환 속도 등이 로봇청소기 구입을 결정할 만한 성능은 아님. 품질요소란 말 그대로 해당 product의 구매의사를 좌우할 수 있는 요소에 해당됨.

로봇청소기 QAS 작성사례에서의 문제점_2

- 사용자가 정해놓은 청소주기가 도래하면 대기 상태인 로봇청소기의 휠 구동부는 1초이내 3km/h까지 가속을 완료해야 한다.
 - 사용자가 완충 및 전원이 Off 된 상태의 로봇 청소기의 전원 버튼을 동작시켜 전원을 On 했을 경우, 로봇 청소기는 10 초 이내에 내부 스피커를 통한 소리를 발생시켜 청소 준비가 되었음을 알린다.
 - Normal 청소 모드 중 먼지 입자의 농도가 30ppm 이상인 경우 모터의 가동력을 500ms이내에 normal mode의 120%로 증가 시킨다.
 - 로봇청소기 이동 구간에 장애물이 있는 경우 1초 이내에 장애물을인식하고 이동방향을 변경해야 한다.
-
- **비기능적 요구사항이 무엇인지 이해하고 있는가?**
 - 청소로봇이 장애물을 인지하여 우회하거나 정지하는 행위, 배터리 소진시 스스로 충전기로 돌아가는 행위, 먼지 농도에 따라 상이한 power를 내는 등은 청소로봇이 제공하는 지엽적인기능적 요구사항에 해당됨. 숫자로 표현된다고 해서 모두 비기능적 요구사항이 되는 것은 아님.

로봇청소기 QAS 작성사례에서의 문제점_3

- 일반 가정집 환경에서 사용자가 로봇 청소기의 표준 자동 청소 기능을 동작시킬 경우, 자동 청소 종료 시 가용 청소 영역의 95% 이상이 청소되어야 한다.
- 사용자가 일반적인 상황에서 자동 청소 기능을 시작 했을 경우, 로봇 청소기는 10분안에 전체 청소 대상 영역에 대해 청소를 완료 하여야 한다
- 먼지가 많은 사용 환경에서 먼지가 거리감지 센서를 오염시켰을 경우에도 거리를 정상적으로 인지 하여 장애물을 회피해야 함

- Measure의 기준이나, Response Measure가 명확한가?

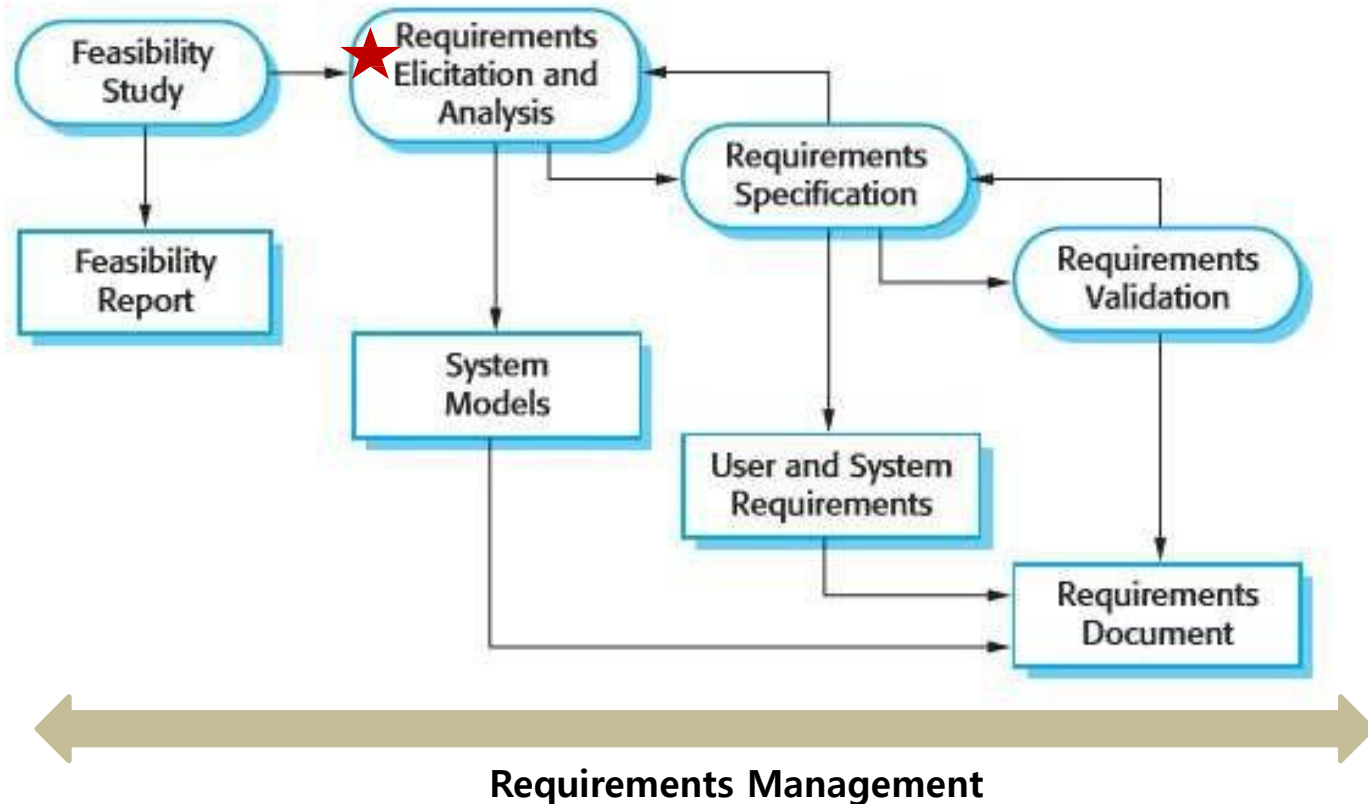
- 형용사로 기술된 response measure 혹은 청소 대상 환경에 대한 모호한 언급 등..

로봇청소기 QAS 작성사례: Good QAS

- “맵이 이미 로봇청소기에 저장된 상태에서, 사용자가 꼼꼼청소 모드를 선택하여 구동시킬 경우, 로봇청소기가 10평짜리 거실 청소를 완료하는 데 소요되는 시간은 15분 미만이다.”

→ HW에 탑재된 SW의 Quality, 그 중에서도 performance 관련 expectation level을 설정할 때, HW의 성능 요소까지 포함된 결과치라는 것이 고민스러운 부분...

QAW in RE Process: Elicitation



Quality Attribute Workshop (QAW)

- Quality Attribute Workshop (QAW)

- Facilitated method
 - System-centric
 - Used before the software architecture has been created
- Engages system stakeholders early in the life-cycle
- Reveals the driving quality attribute requirements of a software-intensive system
 - Scenario-based

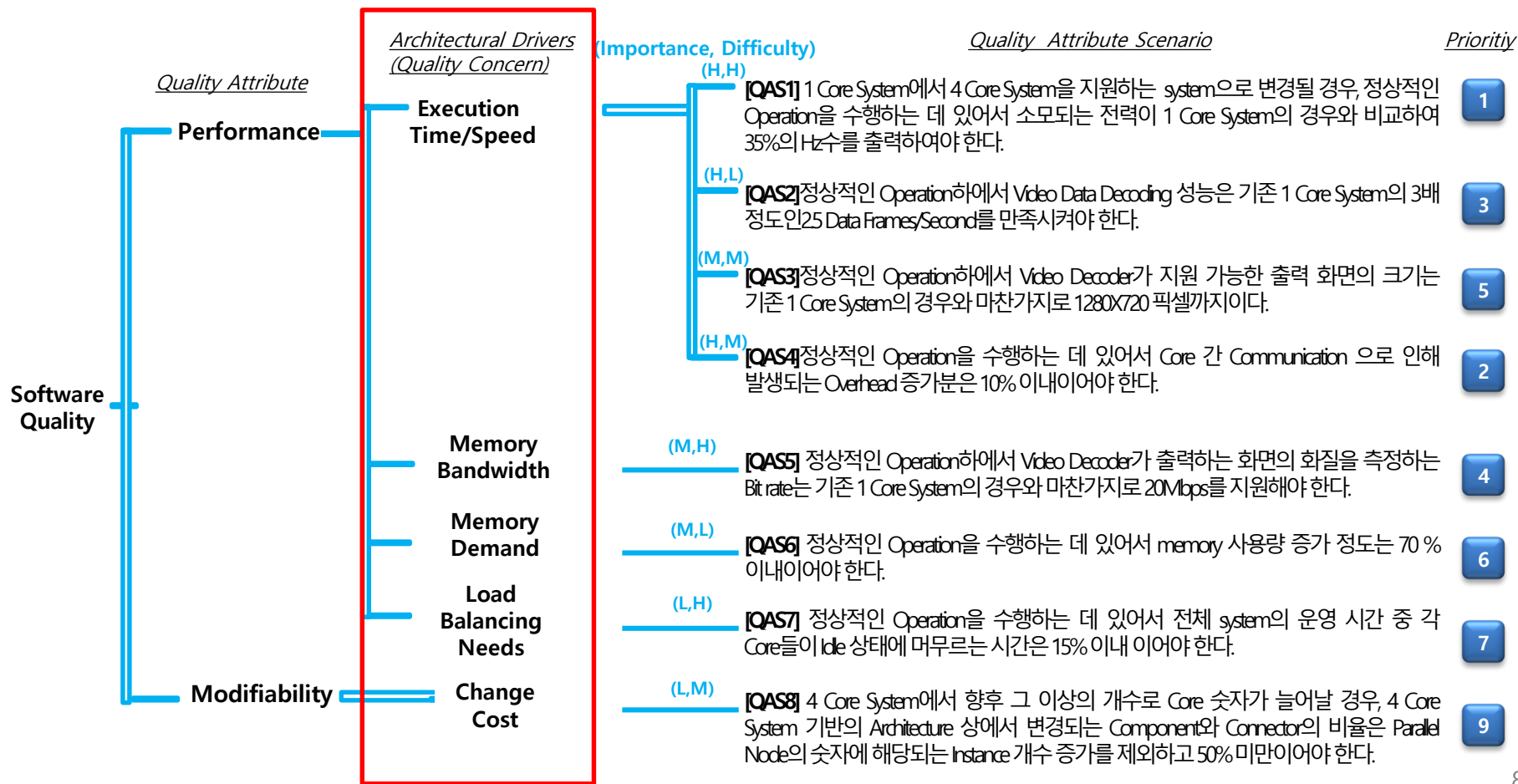
- Outputs of a QAW

- **Quality attribute requirements** for the system, documented as refined and prioritized QAS.
- The quality attribute scenarios can then be used as the basis for designing the software architecture for the system.

The Traditional QAW Steps

1. QAW Introduction
2. Business/Mission Presentation
3. Architectural Plan Presentation
4. Identification of **Architectural Drivers**
5. Scenario Brainstorming
6. Scenario Consolidation
7. Scenario Prioritization
8. Scenario Refinement

Architectural Drivers: Example



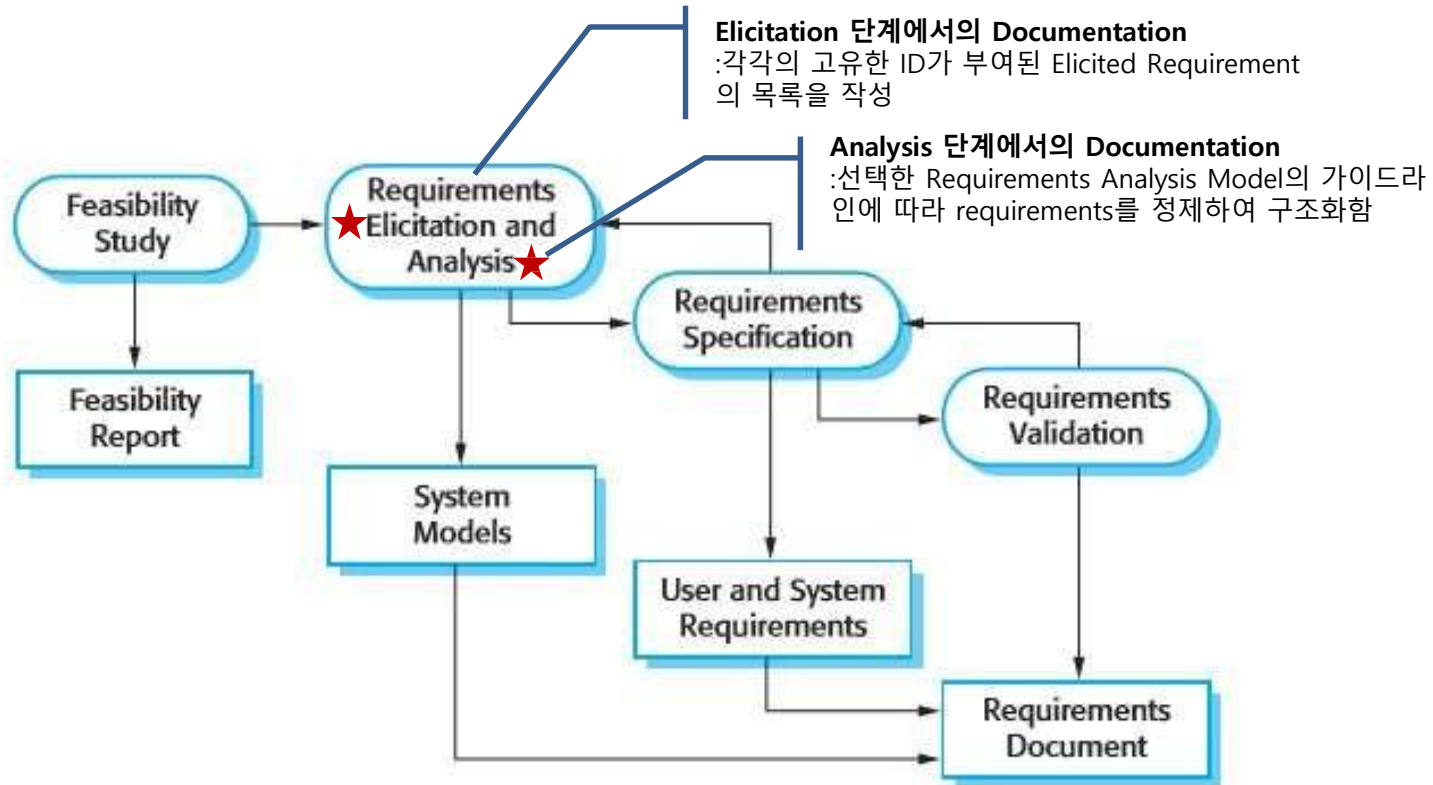
1. Mini-QAW Introduction
2. Introduction to Quality Attributes, Quality Attributes Taxonomy
3. Scenario Brainstorming
 - “Walk the System Properties Web” activity
4. Raw Scenario Prioritization
 - Dot voting
5. Scenario Refinement
 - While time remains, remainder is homework
6. Review Results with Stakeholders



Part 04

Requirements Specification

Different Documentation Activities in Requirements Engineering Process





Software Requirements Specification(SRS)

- The software requirements document is the **official statement** of what is required of the system developers.
 - Should include both a definition of **user requirements** and a specification of the **system requirements**
 - NOT a design document.
 - As far as possible, it should set of **WHAT** the system should do rather than **HOW** it should do it.
- The goal of requirements engineering:
 - *"Not to write the perfect requirements specification, but create the best possible product at the right time"*

Purposes of SRS

- How do we communicate the Requirements to others?
 - It is common practice to capture them in an SRS
 - But, an SRS doesn't need to be a single paper document
- Purpose
 - **Communication**
 - Explains the application domain and the system to be developed
 - **Contractual**
 - May be legally binding!
 - Expresses agreement and a commitment
 - **Baseline for evaluating the software**
 - Supporting testing, V&V
 - "Enough information to verify whether delivered system meets requirements"
 - **Baseline for change control**

- **Software Requirements Specification** should address:

- **Functionality**

- What is the software supposed to do?

- **External interfaces**

- How does the software interact with people, the system's hardware, other hardware, and other software?
 - What assumptions can be made about these external entities?

- **Required performance**

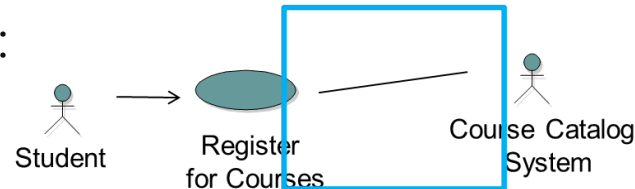
- What is the speed, availability, response time, recovery time of various software functions, and so on?

- **Quality attributes**

- What are the portability, correctness, maintainability, security, and other considerations?

- **Design constraints imposed on an implementation**

- Are there any required standards in effect, implementation language, policies for database integrity, resource limits, operating environment(s) and so on?



SRS Should Not Include

- **Designs**

- Requirements and designs have different audiences
- Analysis and design are different areas of expertise
- Except where application domain constrains the design
 - E.g., limited communication between different subsystems for security reasons

Requirements Document Variability

- Natural languages
- Structured specifications
- Form-based specifications
- Tabular specifications

- **Lack of clarity**

- Precision is difficult without making the document difficult to read.

- **Requirements confusion**

- Functional and non-functional requirements tend to be mixed-up.

- **Requirements amalgamation**

(뭉뚱그려진 requirements 문장들)

- Several different requirements may be expressed together.

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*

3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Example : insulin pump software system

Structured Specifications

- **An approach to writing requirements where**
 - The freedom of the requirements writer is limited and
 - Requirements are written in a standard way.
- **This works well for some types of requirements such as embedded control system.**
 - But, sometimes too rigid for writing business system requirements

Form-based Specifications

- **Specification includes information in a form format :**

- Definition of the function or entity
- Description of inputs, source, outputs, and destination
- Description of the action to be taken
- Pre and post conditions (if appropriate)
- The side effects (if any) of the function

- **Requirements for the insulin pump software system**

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: safe sugar level.

Description

Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2); the previous two readings (r0 and r1).

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose—the dose in insulin to be delivered.

Destination Main control loop.

Action

CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requirements

Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition

The insulin reservoir contains at least the maximum allowed single dose of insulin.

Post-condition r0 is replaced by r1 then r1 is replaced by r2.

Side effects None.

Tabular Specification

- Particularly useful when you have to **define a number of possible alternative courses of action**
 - For example, the insulin pump systems bases its computations on the rate of change of blood sugar level, and the tabular specification explains how to calculate the insulin requirement for different scenarios.
- **Requirements for the insulin pump software system**

Condition	Action
Sugar level falling ($r2 < r1$)	CompDose = 0
Sugar level stable ($r2 = r1$)	CompDose = 0
Sugar level increasing and rate of increase decreasing ($(r2 - r1) < (r1 - r0)$)	CompDose = 0
Sugar level increasing and rate of increase stable or increasing ($(r2 - r1) \geq (r1 - r0)$)	CompDose = round $((r2 - r1)/4)$ If rounded result = 0 then CompDose = MinimumDose

Variations in Requirements Specification Style

- Business Application Domain : 최대한 자세히 줄글 형태로 풀어서 작성하는 스타일을 선호함
 - Example1: 은행 환율등록 시스템(특이사항: CBD방법 적용으로 인한 Variation 포함)

Example1: 은행 환율등록 시스템 (특이사항: CBD방법 적용으로 인한 Variation 포함)

1. 환율등록

1.1 개략적인 설명

본점 Teller 가 환율고시작업을 위하여 TELERATE 또는 REUTER 를 통해서 입수된 기준환율(USD)을 이용하여 각 통화별 대미환산율(CROSS 율)을 입력하는 환율등록작업을 수행한다. 이 외에도 등록완료작업, 등록완료해제작업을 수행하거나, 각종환율기본자료(Spread, 기간별 고시/비고시 Libor)등록작업 등의 외환업무를 수행할 수 있도록 한다.

2. 이벤트의 흐름

2.1 기본 흐름

본 유스케이스는 본점 Teller 가 환율등록(완료, 완료해제)및 기간별 고시/비고시 LIBOR 등록, Spread 등록작업을 하기 위해 해당되는 작업을 선택하면서 시작된다.

1. 환율등록선택

본점 Teller 는 메뉴에서 환율등록을 선택한다.(A1)

시스템은 환율등록 화면을 display 한다.

2. 취급점번호 체크

시스템은 취급점번호를 체크해서 취급점번호가 국제팀(1955), 외환업무팀(4033), 법인영업부(9724)가 아닌경우, E1 수행한다.

3. 기간별 고시/비고시 Libor 등록여부 확인

시스템은 기간별 고시/비고시 Libor 등록여부를 확인하여 해당일자의 Libor 가 등록되지 않은 경우, A1 수행한다.

4. 선처리 작업을 위해 “선처리 “ 유스케이스를 포함한다.

5. 업무개시일자의 익영업일 검색

시스템은 선처리작업의 결과로 검색된 업무개시일자와 Current Date 를 비교한다.

5.1 Current Date 가 업무개시일자보다 큰 경우(업무개시전):

업무개시일자의 익영업일을 검색한다.

등록일자 = 업무개시일자의 익영업일

사용 Component : 캘린더

input : 업무개시일자

output : 익영업일자

5.2 Current Date 와 업무개시일자가 동일한 경우(업무개시후):

Variations in Requirements Specification Style

- Business Application Domain : 최대한 자세히 줄글 형태로 풀어서 작성하는 스타일을 선호함
 - Example1: 은행 환율등록 시스템(특이사항: CBD방법 적용으로 인한 Variation 포함)
- Embedded Application Domain: 표 형태의 template+개조식 기술방식을 선호함
 - Example2: DVD Player Controller(table style)
 - Example3: H.264 표준을 따르는 영상 데이터 디코딩(table style + pseudo code에 가까운 depth)

Example2: DVD Player Controller(table style)

Feature Group ID	SETUP.AUDIO		
그룹설명	Audio 출력에 관한 설정을 처리하는 module		
고려사항			
세부사양			
Requirement ID	SRS_FEAT_4		시스템 적용여부
Feature ID	SETUP.AUDIO.DIGITAL-OUTPUT		
설명	Digital Audio 출력 방식을 설정한다.		
Pre-condition	1.Stop 상태여야 한다.		
Post-condition	Disc play 시 설정된 방식의 Digital Audio 가 출력된다.		
개발 우선순위	A		
기본동작	BF1	Triggering Event	
	BF1.1	사용자에 의해서 Menu/Setup/Audio/Digital Output 항목을 선택한다.	Y
	BF2	사용자가 원하는 출력 방식을 방향키를 이용해서 선택한다.	Y
	BF3	다음 중 한 Step 을 수행한다.	
	BF3.1	사용자가 PCM 을 선택하고 Enter 를 누를 경우 [AF1]	Y
	BF3.2	사용자가 Bitstream 을 선택하고 Enter 를 누를 경우 [AF2]	Y
	BF4	Digital output 설정 값을 EEPROM 에 저장한다.	Y
대체동작	AF1	사용자가 PCM 을 선택하고 Enter 를 누를 경우 Digital output 설정 값을 PCM 으로 변경한다.	Y
	AF2	사용자가 Bitstream 을 선택하고 Enter 를 누를 누를 경우 Digital output 설정 값을 Bitstream 으로 변경한다.	Y
예외동작			
품질요구사항	QR1	변경 후 재생 시 Noise 가 발생되지 말아야 한다.	Y

Example3: H.264 표준을 따르는 영상 데이터 디코딩(table style + psuedo code에 가까운 depth)

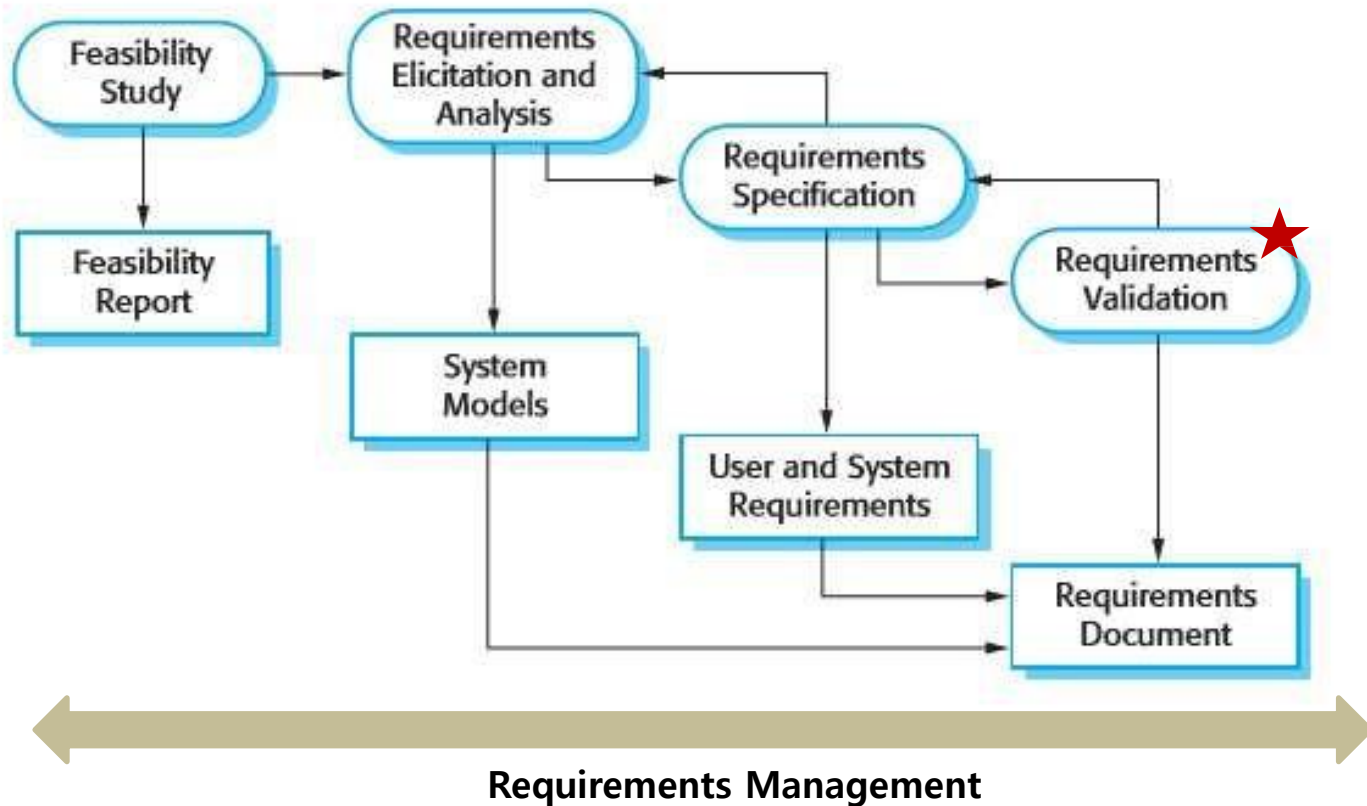
Requirement ID	H.264 표준을 따르는 영상 데이터 디코딩	
설명	H.264 규약에 따라 동작하는 Video Decoder 가 입력되는 Video Data 의 압축을 풀어 화면상에 출력하기까지 시스템의 동작을 기술한다.	
Pre-condition	입력 Video Data 가 DRAM 상에 load 되어 있는 상태이다.	Parallelism
Post-condition	입력된 Video Data 가 출력 Buffer 상에 모두 출력된 상태이다.	
개발우선순위	1	
기본동작	BF1 비디오 디코더에게 디코딩 메시지가 도착한다.	N
	BF2 Input Data Preprocessing 을 시작한다.	N
	BF2.1 Macro Block(MB) 정보를 저장하는 자료구조 버퍼를 초기화한다.	N
	BF2.2 NAL 데이터들을 입력 받아, FileBuffer[]에 저장한다.	N
	BF2.3 FileBuffer[]에서 한 개의 NAL 데이터를 작업 Buffer 로 복사한다.	N
	BF2.4 해당 NAL 데이터의 Header type 값을 parsing 한다 만약에 입력된 NAL 데이터 자체에 문제가 있거나 해당 데이터의 Header Type 이 유효하지 않으면, EF1 로 분기한다.	N
	BF2.4.1 [NAL Header Type == PPS]	N
	BF2.4.1.1 Picture 정보를 picture_parmeter_set 버퍼에 저장한다.	N
	BF2.4.2 [NAL Header Type == IDR]	N
	BF2.4.2.1 IDR Picture 정보를 해당 버퍼에 저장한다.	N
	BF2.4.3 [NAL Header Type == PPS]	N
	BF2.4.3.1 sequence 정보를 seq_parmeter_set 버퍼에 저장한다.	N
	BF2.4.4 [NAL Header Type == Slice]	N
	BF2.4.4.1 Slice 를 구성하는 MB 에 대한 영상 복원을 시작한다.	N
	BF3 Entropy Decoding 을 시작한다.	FP_01
	BF3.1 처리하고자 하는 MB 의 정보를 SPS, PPS, IDR 버퍼에서 읽어온다.	DP_01
	BF3.2 MB 의 picture type 을 parsing 한다.	DP_01
	BF3.2.1 [MB type == Intra-Picture]	DP_01
	BF3.2.1.1 해당 MB 정보를 ipredmode[] 버퍼에 복사한다.	DP_01



Part 05

Requirements Validation

QAW in RE Process: Elicitation



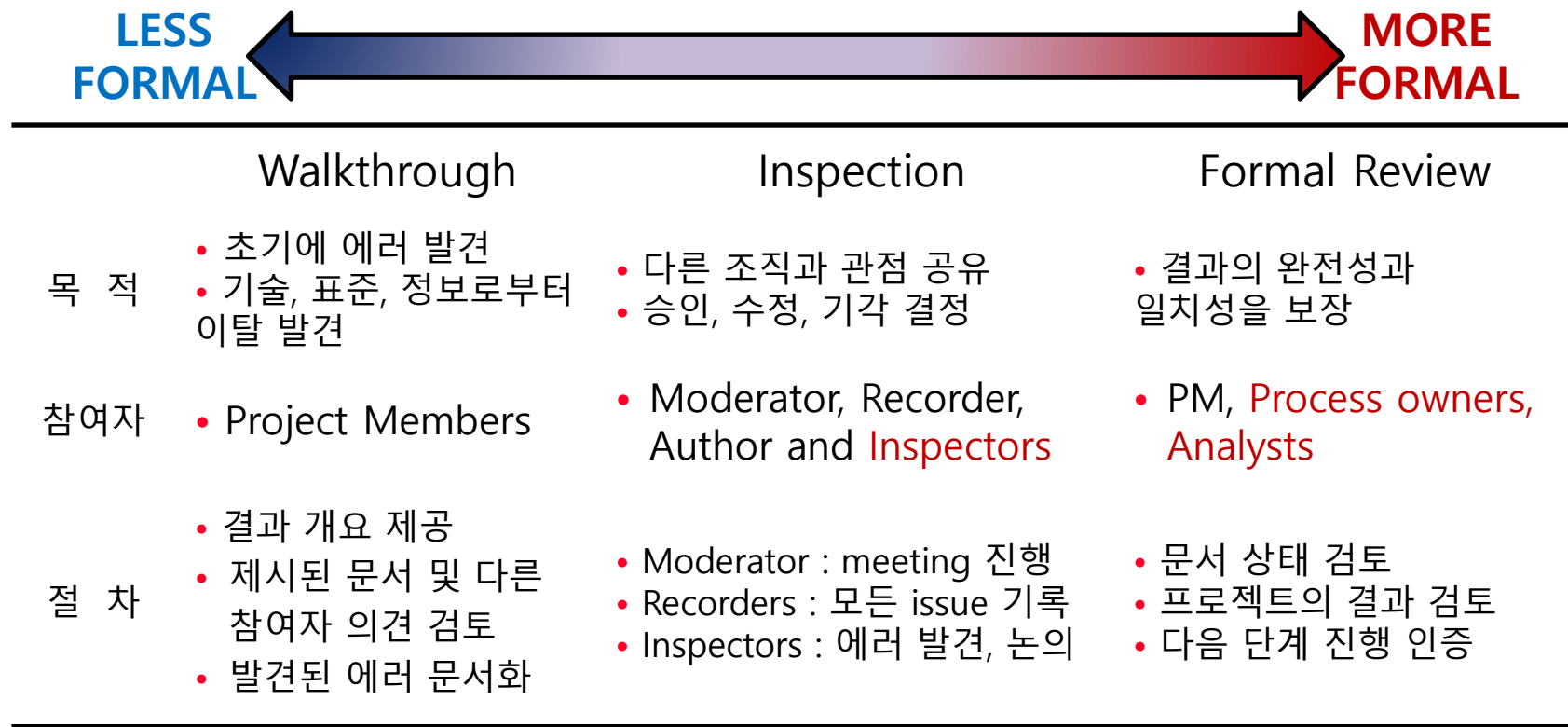
Requirements Validation

- Demonstrate whether *the requirements we defined* are **what the customer really wants**
- Requirements validation checks:
 - Validity: Does the system provide the functions which *support the customer's needs well?*
 - **Correctness** : Are these the *right requirements* to achieve goal X
 - **Consistency** : Are there any requirements *conflicts?*
 - **Completeness** : Are *all functions* required by the customer included?
 - Feasibility: Can the requirements be *implemented with available budget and technology?*
 - Verifiability: Can the requirements be checked?

Requirements Validation Techniques

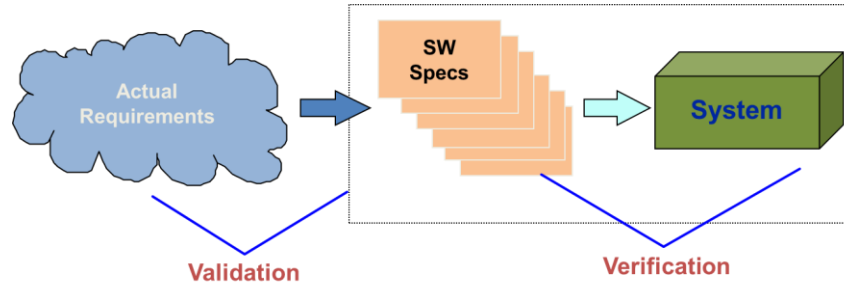
- **Requirements reviews**
 - Systematic manual analysis of the requirements.
- **Prototyping**
 - Using an executable model of the system to check requirements.
- **Test-case generation**
 - Developing tests for requirements to check testability.

Requirements Reviews

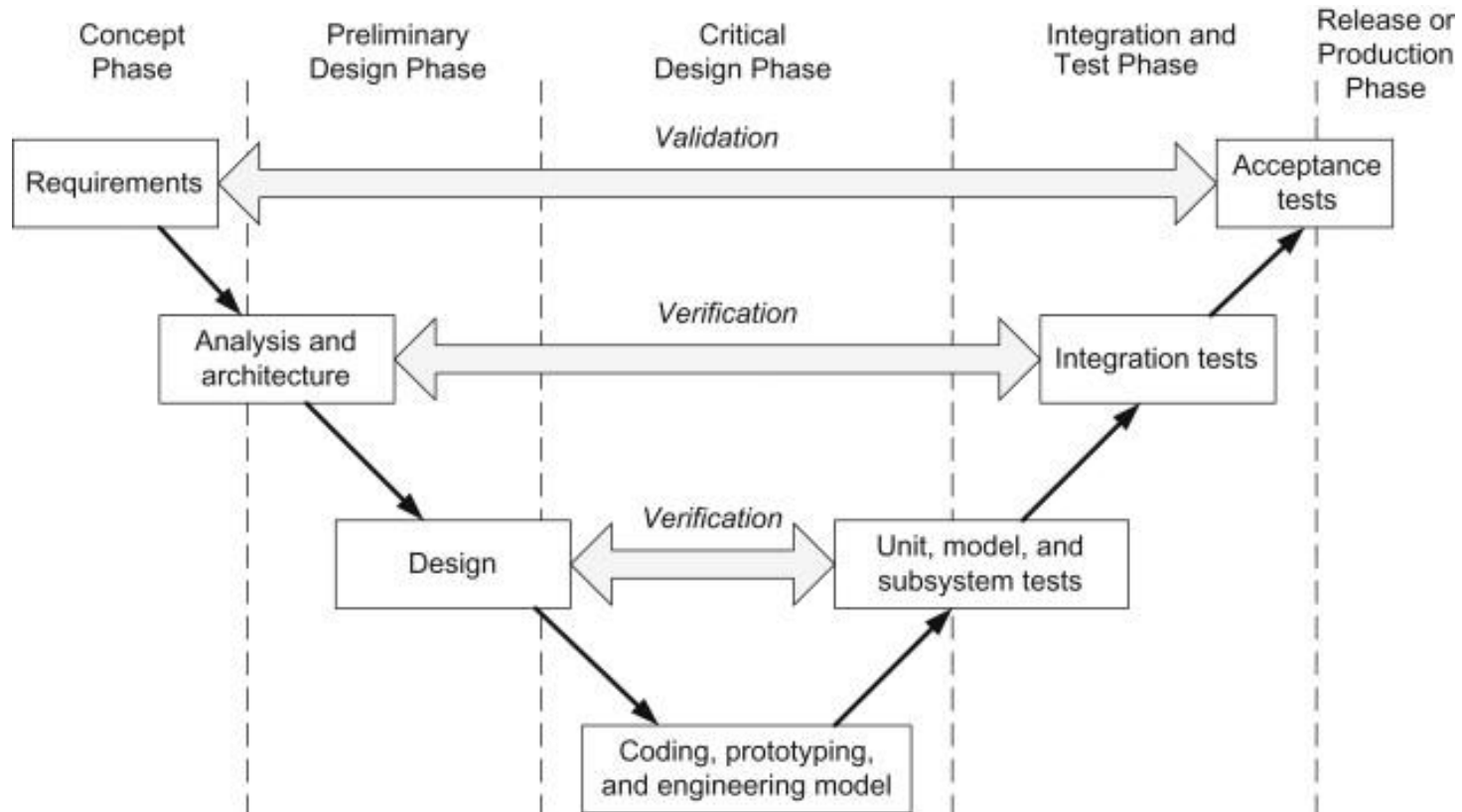


Verification and Validation in SDLC

- **Validation:** "Does the software system **meets the user's real needs?**"
 - Are we building the **right software**?
 - Does our design meet the spec?
 - Does our implementation meet the spec?
 - Does the delivered system do what we said it would do?
 - Are our requirements models consistent with one another?
- **Verification:** "Does the software system **meets the requirements specifications?**"
 - Are we building the **software right** ?
 - Does our problem statement accurately capture the real problem?
 - Did we account for the needs of all the stakeholders?



V-Model of V&V Activities in SDLC

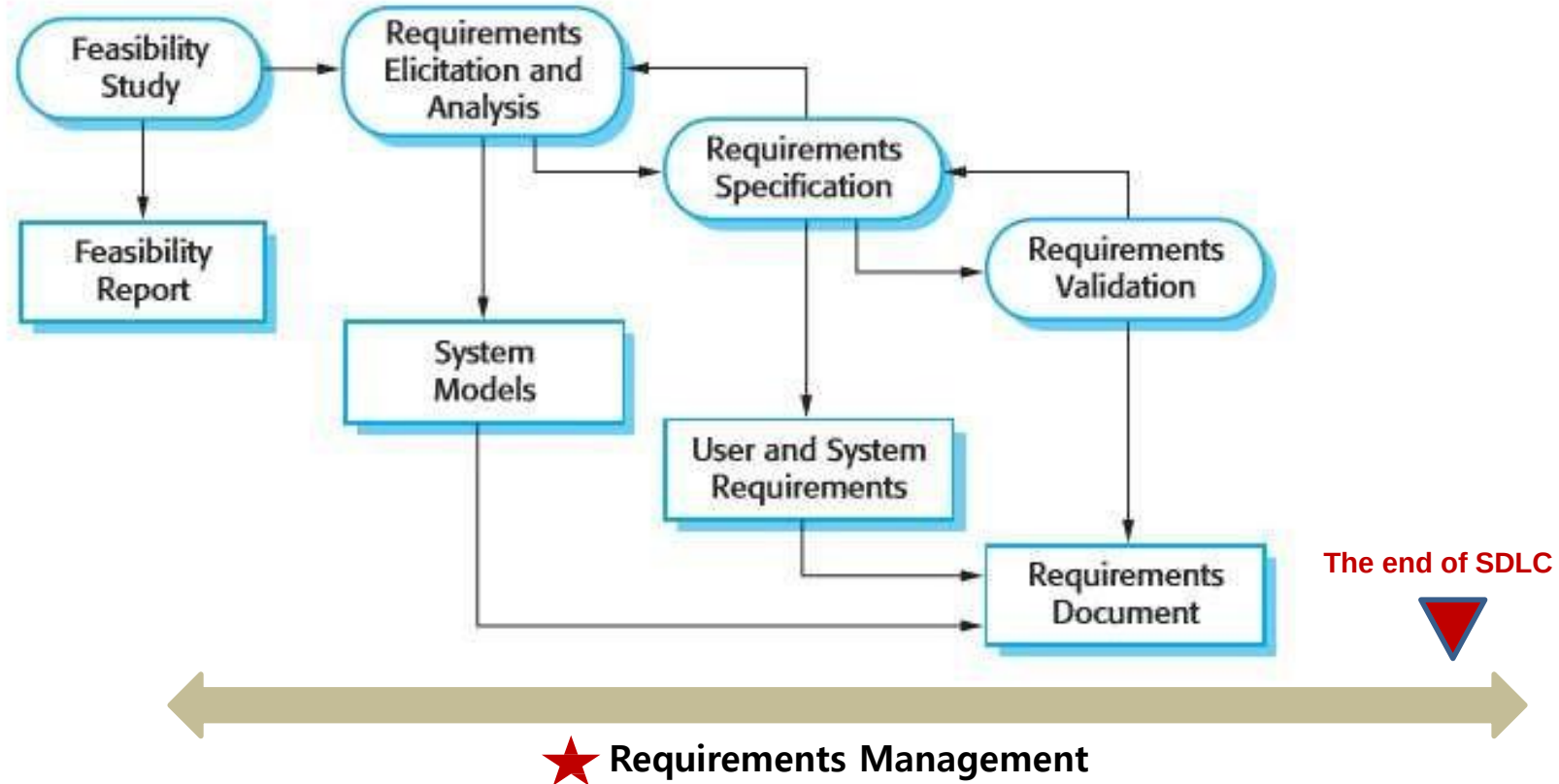


A decorative blue sidebar on the left side of the slide. It features several overlapping circles and semi-circles. Some are solid blue, while others have patterns of white dots or diagonal lines. There are also white vertical lines and small circles scattered throughout the design.

Part 06

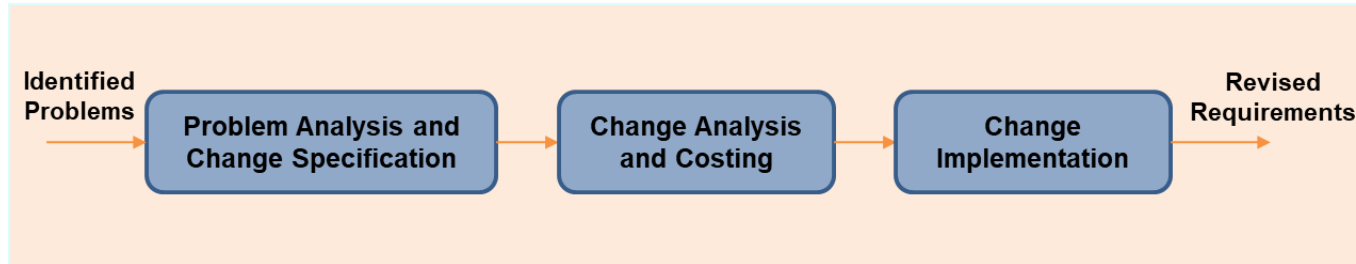
Requirements Management

Requirements Engineering Process



Requirements Change Management

- The process of managing requirements change during the RE process and the overall system development
 - Requirements are inevitably incomplete and inconsistent.
 - New requirements emerge during the process, as business needs change and a better understanding of the system is developed.

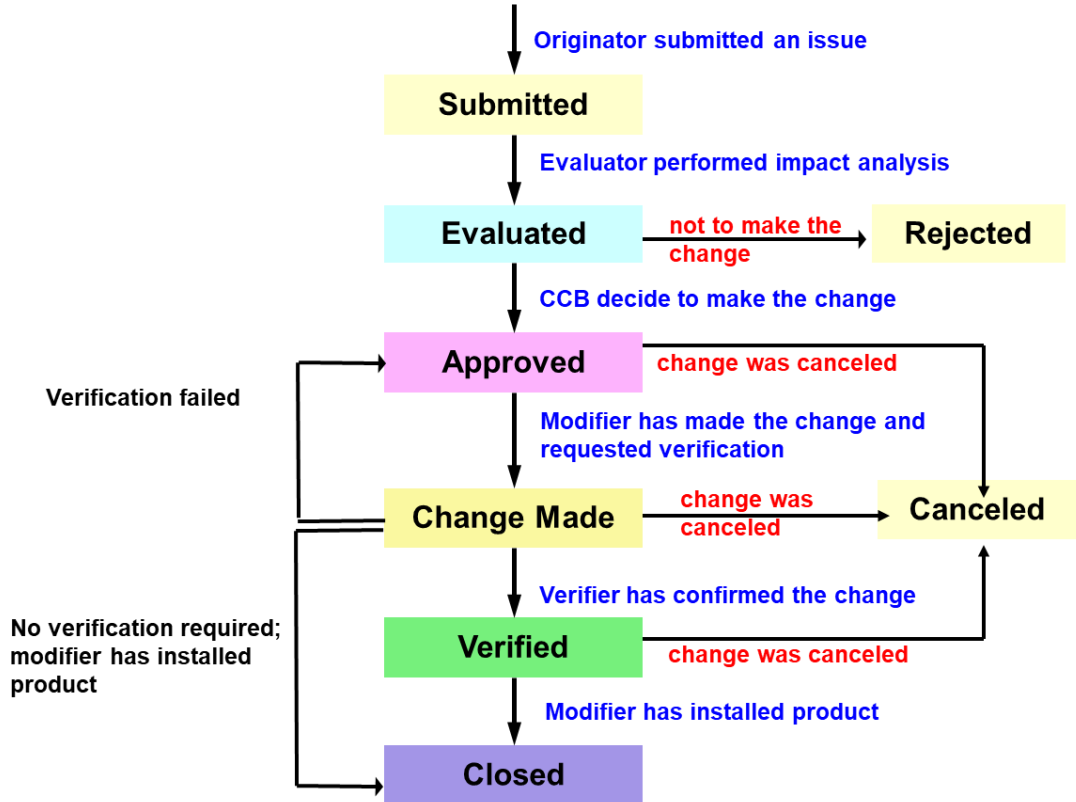


- **Traceability** is the heart of requirements management.
 - Source ↔ Requirements ↔ Design ↔ Code

Change Management Process

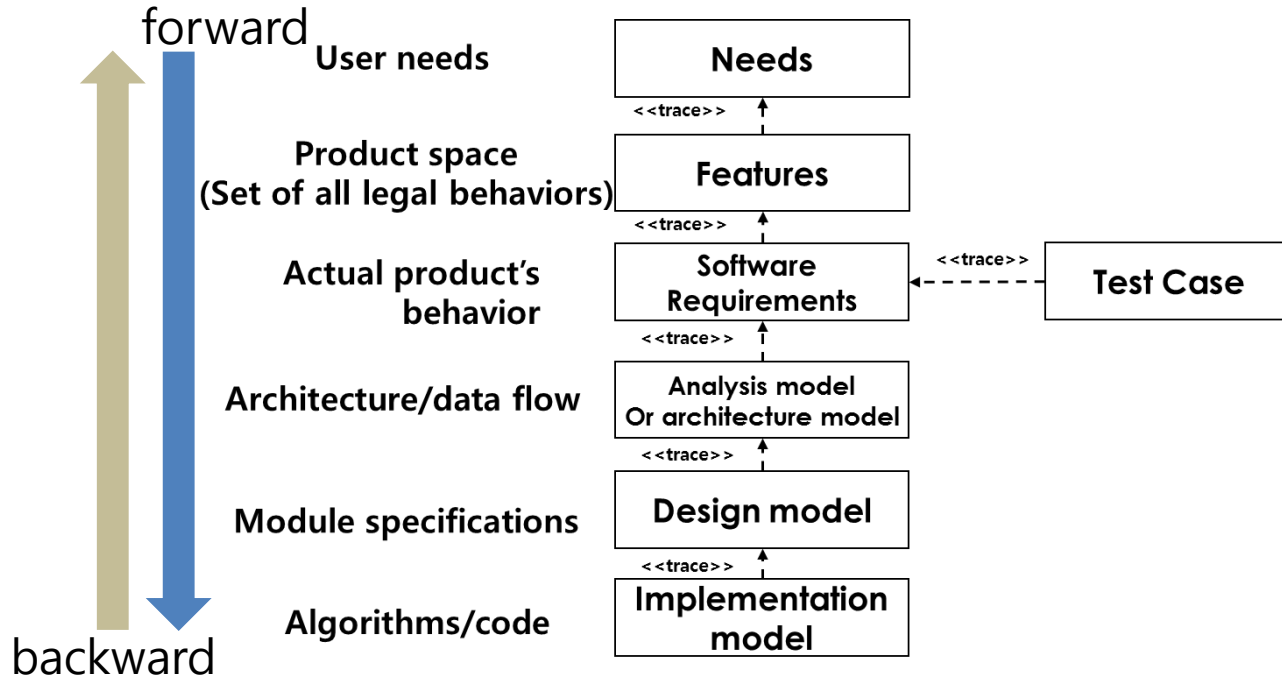
- **Change Management Process**

- All proposed changes are submitted formally as **change requests**
- A **review board** reviews these periodically and decides which to accept



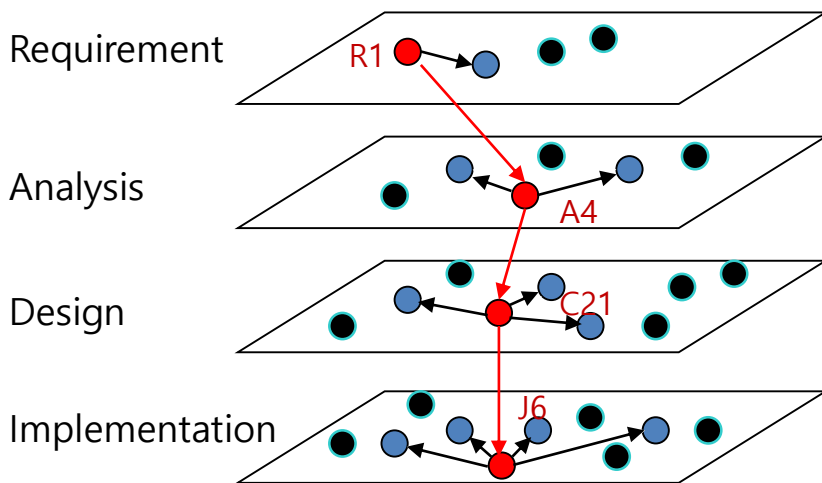
Requirements Traceability

- When a high level artifact derives a refined artifact, Traceability link should be generated between two artifacts.

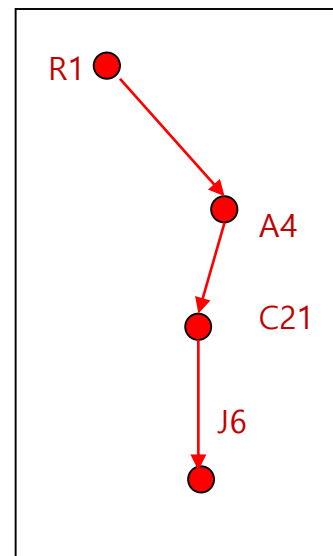


Traceability Policy

- Horizontal Traceability Link & Vertical Traceability Link

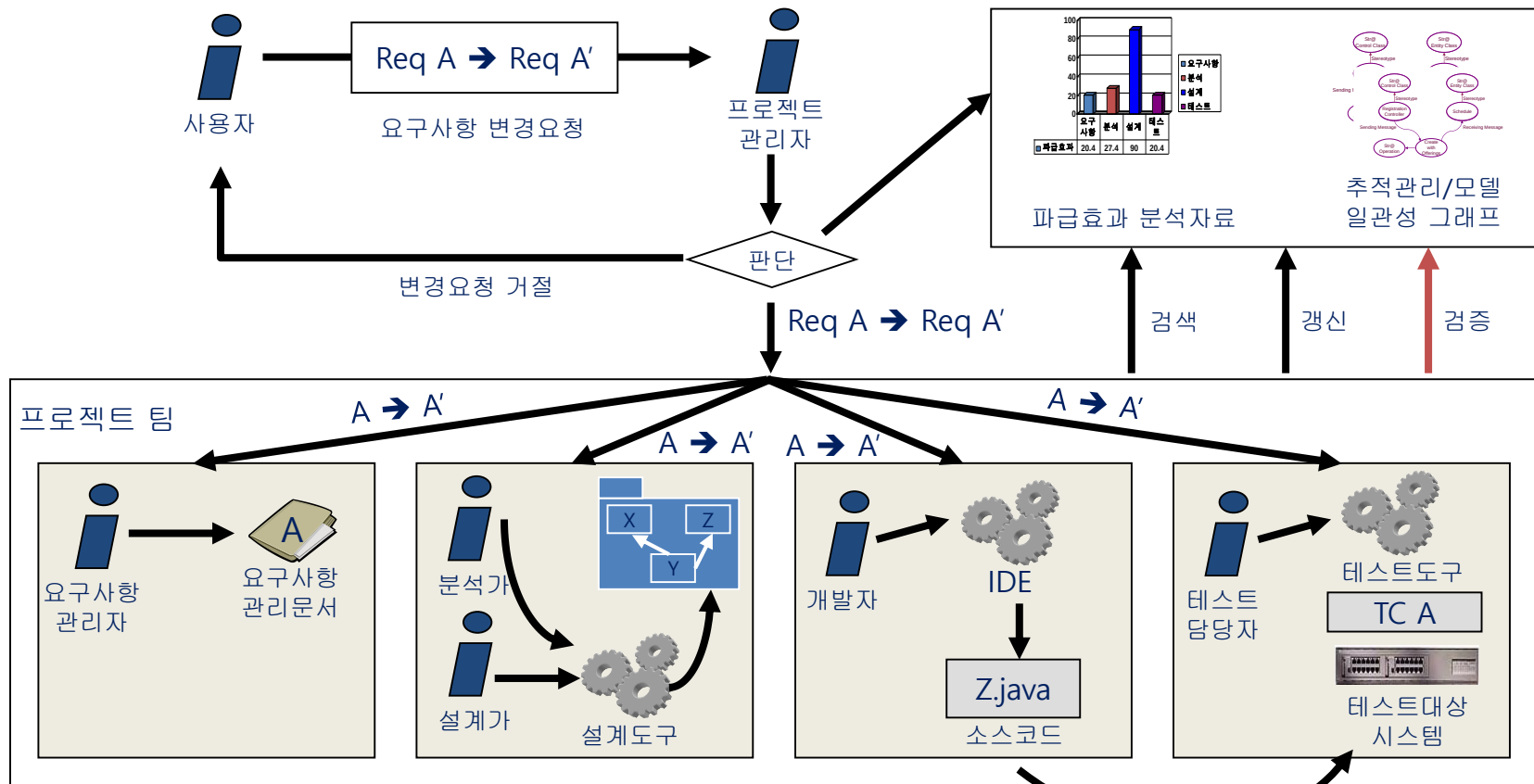


(a) A Trace Model with Vertical Traceability Link and Horizontal Traceability Link



(b) Focusing Vertical Traceability

Benefit of Req. Traceability: Change Impact Analysis



Traceability Difficulties

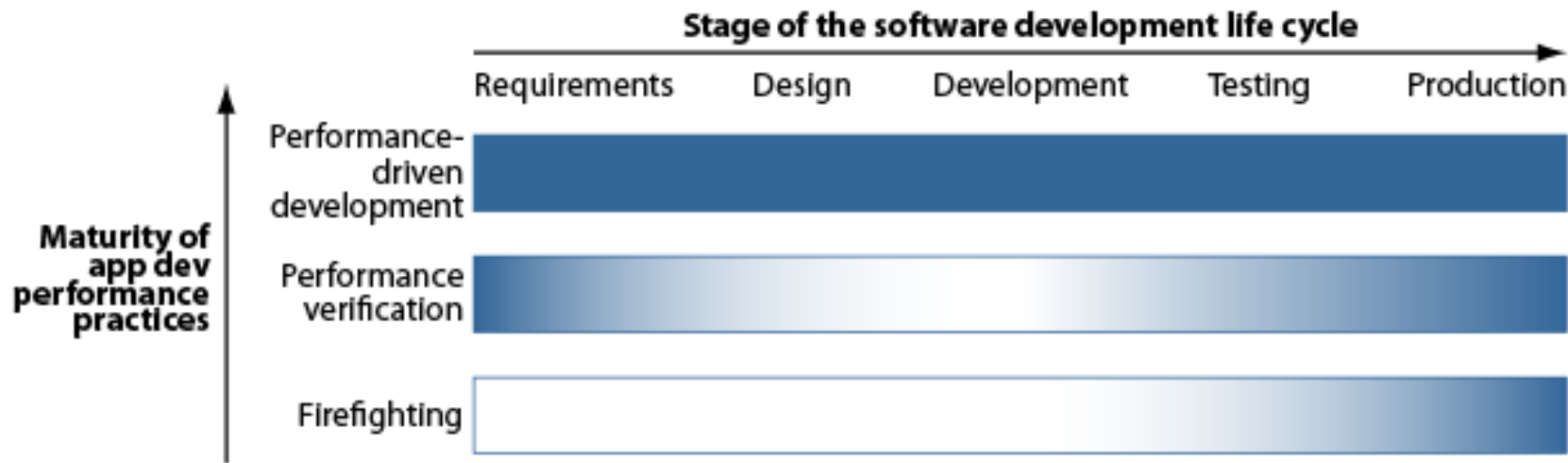
- **Cost**
 - Very little automated support
 - Full traceability is very expensive and time-consuming
- **Delayed gratification**
 - The people defining traceability links are not the people who benefit from it
 - Development vs. V&V
 - Much of the benefit comes late in the lifecycle
 - Testing, integration, maintenance
- **Size and diversity**
 - Huge range of different document types, tools, decisions and responsibilities
 - No common schema exists for classifying and cataloging these
 - In practice, traceability concentrates only on baselined requirements

A decorative blue sidebar on the left side of the slide. It features several overlapping circles and semi-circles. Some are solid blue, while others have patterns of white dots or diagonal lines. There are also white vertical lines and small circles scattered throughout the design.

Part 07

Wrap-up

Just Enough Process!!



Forrester, February 2006, Best Practices “Performance-Driven Software Development”

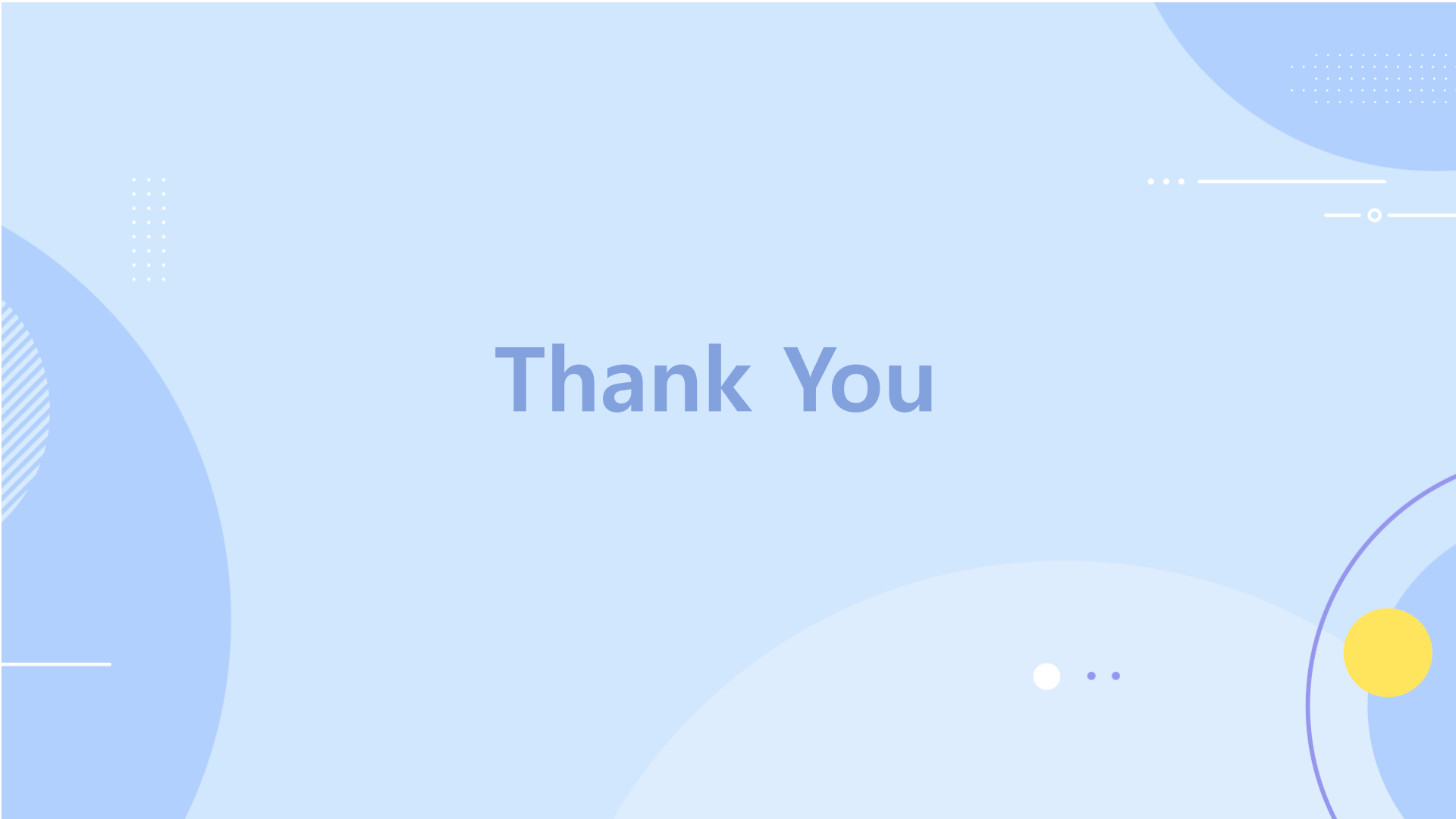
RE Process Tailoring is Needed..

- 삼성전자 DAV
 - [SW 요구사항서\(SRS\) 작성지침 삼성전자 DAV](#)

Tips for Tailoring

빈 양식(Template)만으로는 아무것도 가이드할 수 없음.

해당 도메인의 대표적인 요구사항 타입을 정의하고 이에 대한 STEREOTYPED EXAMPLE을 제시해야 함.

The background is a light blue gradient. It features several abstract geometric elements: a large blue circle on the left with a white horizontal line; a blue circle in the top right with a white dotted pattern; a blue circle in the bottom right with a yellow circle inside it; and a light blue circle at the bottom center with a white dot and two blue dots. There are also white dotted patterns in the top right and middle left, and a white horizontal line with a dot in the top right.

Thank You